

JUDUL DISINI

TESIS

**Karya tulis sebagai salah satu syarat
untuk memperoleh gelar Magister dari
Institut Teknologi Bandung**

**Oleh:
ZIADAN QOWI
NIM: 23524019
Program Studi Magister Informatika**



**INSTITUT TEKNOLOGI BANDUNG
2026**

JUDUL DISINI

Oleh

Ziadan Qowi

NIM 23524019

(Program Studi Magister Informatika)

Institut Teknologi Bandung

Menyetujui

Tim Pembimbing

Tanggal

Ketua

(Ir. Robithoh Annur, S.T., M.Eng., Ph.D.)

NIP.

Anggota

(Dr. Phil. Eng. Hari Purnama, S.Si., M.Si)

NIP.

ABSTRAK

Tuliskan abstrak di sini...

ABSTRACT

Write your abstract here...

Daftar Isi

LEMBAR PENGESAHAN	i
ABSTRAK	ii
ABSTRACT	iii
Daftar Isi	vi
Daftar Gambar	vii
Daftar Tabel	viii
DAFTAR SINGKATAN DAN ISTILAH	ix
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	6
I.3 Tujuan Penelitian	6
I.4 Kontribusi Penelitian	7
I.5 Hipotesis dan Premis	8
I.6 Batasan Masalah	9
I.7 Metodologi Penelitian	10
I.8 Sistematika Penulisan	12
II TINJAUAN PUSTAKA DAN LANDASAN TEORI	14
II.1 IoT, Keamanan, dan Kebutuhan Kontrol Akses	14
II.2 Model Kontrol Akses dan Relevansi ABAC pada IoT	15
II.3 Dasar Pemilihan Atribut Kebijakan ABAC	17
II.4 Edge/Fog Computing, Attribute Caching, dan Freshness	20
II.5 Threat Model dan Metrik Evaluasi Keamanan	21
II.6 Optimasi dan Metaheuristik untuk Konfigurasi ABAC Edge	23
II.7 Dasar Pemilihan Konfigurasi Uji dan Skenario Workload	25

II.7.1	Dasar Pemilihan Konfigurasi Uji	25
II.7.2	Dasar Penentuan Skenario Dinamika Workload	27
II.8	Penelitian Terkait	30
II.9	Sintesis Literatur dan Posisi Penelitian	33
III	METODOLOGI PENELITIAN	35
III.1	Jenis dan Pendekatan Penelitian	35
III.2	Alur Umum Penelitian	36
III.3	Arsitektur Sistem yang Diusulkan	39
III.4	Model ABAC yang Digunakan	43
III.4.1	Komponen ABAC	43
III.4.2	Policy ABAC	43
III.4.3	Oracle Decision dan Candidate Decision	43
III.5	Model Cache Atribut dan Freshness	43
III.6	Threat Model dan Asumsi Keamanan	43
III.7	Desain Skenario Workload	43
III.8	Konfigurasi Eksperimen	43
III.8.1	Cloud-Only ABAC	43
III.8.2	Edge Fresh ABAC	43
III.8.3	Static Cache ABAC	43
III.8.4	Adaptive-TTL ABAC	43
III.8.5	Random Search ABAC	43
III.8.6	GWO-Optimized ABAC	43
III.9	Representasi Solusi Optimasi	43
III.10	Fungsi Fitness	43
III.11	Algoritma Optimasi	43
III.11.1	Random Search	43
III.11.2	Grey Wolf Optimization	43
III.12	Training dan Testing	43
III.13	Metrik Evaluasi	43
III.13.1	Metrik Performa	43
III.13.2	Metrik Kualitas Keputusan	43
III.13.3	Metrik Freshness dan Revocation	43
III.14	Prosedur Eksperimen	43
III.15	Konfigurasi Lingkungan Pengujian	43
III.16	Teknik Analisis Data	43
IV	HASIL DAN PEMBAHASAN	44

IV.1 Implementasi Simulasi pada iFogSim2	44
IV.2 Konfigurasi Lingkungan dan Skenario Pengujian	44
IV.3 Hasil Pengujian dan Analisis	44
IV.4 Pembahasan	44
V KESIMPULAN DAN SARAN	45
V.1 Kesimpulan	45
V.2 Saran	45
DAFTAR PUSTAKA	46

Daftar Gambar

I.1	Prakiraan pasar IoT global dalam miliaran perangkat IoT yang terhubung (Sinha, 2025)	1
III.1	Diagram alir alur umum penelitian	36
III.2	Sequence diagram arsitektur sistem yang diusulkan	41

Daftar Tabel

II.1	Perbandingan penelitian terkait ABAC, caching, dan optimasi pada IoT	32
------	--	----

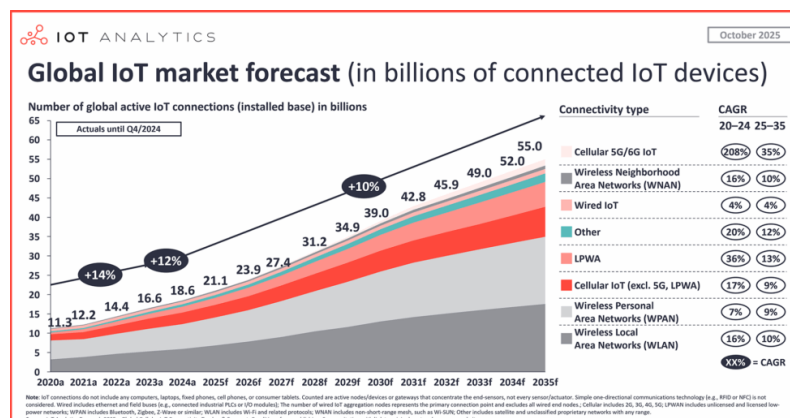
DAFTAR SINGKATAN DAN ISTILAH

Istilah / Sing-	Kepanjangan /	Penjelasan
katan	Asing	

BAB I PENDAHULUAN

I.1 Latar Belakang

Dalam beberapa tahun terakhir, *Internet of Things* (IoT) telah berkembang dari sekadar konsep konektivitas antar perangkat menjadi infrastruktur digital berskala sangat besar yang menghubungkan sensor, aktuator, dan aplikasi untuk mendukung otomatisasi serta pengambilan keputusan di berbagai sektor. Sebuah laporan menunjukkan bahwa pertumbuhan jumlah perangkat IoT pada tahun 2024 telah mencapai hingga 18,6 miliar dan diperkirakan melonjak hingga angka 39 miliar perangkat pada tahun 2030 (Sinha, 2025). Di level aplikasi, sebuah penelitian menunjukkan bahwa IoT telah memperluas fungsi *smart home* dari *monitoring* dasar ke layanan yang lebih berpusat pada kualitas hidup seperti keamanan, penghematan energi, dan dukungan kesehatan (Park dan Han, 2025). Sementara pada sektor industri, *Industrial Internet of Things* (IIoT) berperan nyata dalam meningkatkan efisiensi operasional, kualitas pengambilan keputusan, optimasi biaya, dan keselamatan kerja (Afrin dkk., 2025). Dengan perluasan skala dan dampak lintas domain tersebut, IoT kini bukan lagi teknologi pelengkap, melainkan fondasi transformasi digital yang semakin kritical bagi kehidupan sehari-hari maupun operasi organisasi modern.



Gambar I.1 Prakiraan pasar IoT global dalam miliaran perangkat IoT yang terhubung (Sinha, 2025)

Seiring dengan semakin masif dan heterogennya ekosistem IoT, mekanisme kontrol akses yang digunakan tidak lagi cukup apabila hanya bersifat statis, *coarse-grained*, atau bergantung pada identitas semata, karena perangkat, pengguna, layanan, dan kondisi lingkungan terus berubah secara dinamis. Survei terbaru mengenai model otorisasi pada IoT menegaskan bahwa kebutuhan utama kontrol akses modern mencakup *fine-grained authorization* dan kemampuan *context-aware* untuk mempertimbangkan atribut subjek, objek, aksi, dan lingkungan, serta skalabilitas agar kebijakan tetap dapat diterapkan secara konsisten pada sistem dengan jumlah perangkat dan interaksi yang sangat besar (Díaz dan Mendoza, 2025 dan Ragothaman dkk., 2023). Kebutuhan tersebut menjadi semakin penting pada lingkungan *edge/fog*, karena keputusan akses sering kali harus dibuat dekat dengan sumber data dan dalam waktu yang cepat, sehingga model kontrol akses dituntut tidak hanya akurat dalam kebijakan, tetapi juga mampu beradaptasi terhadap perubahan konteks dan perilaku akses secara *real-time*. Hal ini juga tercermin dalam penelitian tentang *adaptive context-aware access control* pada IoT yang menunjukkan bahwa pengambilan keputusan berbasis konteks diperlukan untuk mencegah akses tidak sah pada lingkungan yang sangat dinamis dan terdistribusi (Kalaria dkk., 2024).

Di antara semua model kontrol akses yang ada, model *Attribute-Based Access Control* (ABAC) adalah salah satu model yang paling relevan untuk lingkungan IoT karena menawarkan skalabilitas tinggi, fleksibilitas besar, dan dinamika yang kuat (Hu dkk., 2014). Dalam model ABAC, permintaan subjek untuk melakukan tindakan pada suatu objek diizinkan atau ditolak berdasarkan sekumpulan atribut yang diberikan kepada subjek, objek, dan lingkungan (Díaz dan Mendoza, 2025). Berbeda dengan model kontrol akses tradisional yang memerlukan penetapan kepemilikan, tingkat keamanan, dan peran secara manual oleh administrator sistem. Selain itu, model ABAC secara efektif menyederhanakan manajemen akses karena jumlah atribut jauh lebih sedikit dibandingkan jumlah subjek/pengguna dan objek/perangkat IoT dalam sistem (Bakhtiary dkk., 2024).

Meskipun ABAC unggul dalam hal fleksibilitas kebijakan, implementasinya pada lingkungan IoT yang heterogen dan *resource-constrained* bukannya tanpa masalah. Efektivitas ABAC sangat bergantung pada penggunaan atribut yang akurat dan mutakhir (Cremonezi dkk., 2023). Namun, atribut-atribut yang diperlukan untuk evaluasi kebijakan umumnya berasal dari sumber eksternal, seperti *Policy Information Point* (PIP) yang berada di *cloud*. Sayangnya, pada skenario dengan permintaan akses yang tinggi, proses pengambilan atribut yang dilakukan berulang setiap kali sebuah kebijakan dievaluasi menciptakan *bottleneck* berupa peningkatan jumlah panggilan ke PIP (*PIP calls*), latensi keputusan yang tinggi, serta *overhead* komunikasi yang

membebani jaringan. Selain itu, ketika atribut-atribut ini disimpan sementara pada *node fog* yang memiliki keterbatasan penyimpanan, timbul pula tambahan terkait biaya menjaga *freshness* dan keamanan atribut tersebut (Cremonezi dkk., 2022). Kondisi ini semakin kritis pada aplikasi IoT yang membutuhkan respons cepat secara *real-time* serta berjalan pada perangkat dengan keterbatasan komputasi, memori, dan daya. Dalam konteks ABAC, proses pengambilan atribut dari sumber yang jauh dari *Policy Decision Point* (PDP) dapat meningkatkan latensi dan menunda proses otorisasi sehingga berpotensi mengganggu kontinuitas layanan (Cremonezi dkk., 2022). Pendekatan *cloud-based access control* juga dilaporkan menimbulkan *overhead* komunikasi, latensi tinggi, dan ketergantungan tinggi pada infrastruktur *cloud* sehingga kurang ideal untuk lingkungan IoT yang dinamis (Kaloria dkk., 2024). Oleh karena itu, berbagai studi menekankan perlunya memindahkan sebagian proses pengambilan keputusan akses, informasi kebijakan, dan/atau *attribute handling* ke lapisan *edge* atau *fog* agar komputasi berlangsung lebih dekat dengan sumber data, menurunkan latensi, mengurangi komunikasi ke *cloud*, dan meningkatkan efisiensi sistem (Frimpong dkk., 2022, Xue dkk., 2025 dan Laroui dkk., 2021).

Namun, ketika informasi identitas atau atribut akses disimpan secara lokal di *edge* untuk mempercepat evaluasi, muncul tantangan baru terkait *freshness* atau kesegaran atribut. Wang dkk. (2025) menunjukkan bahwa *edge-based authentication and authorization* dapat menempatkan PDP dan PEP di *edge* dengan data *Identity and Access Management* (IAM) yang disimpan secara lokal untuk mendukung kontrol akses dengan latensi yang rendah. Akan tetapi, penelitian tersebut juga menegaskan adanya *trade-off* antara *attribute freshness* dan keamanan. Dalam kondisi konektivitas yang terputus-putus, *local caching* memang diperlukan, tetapi dapat menimbulkan risiko keamanan akibat *stale attributes*, misalnya ketika data sudah dicabut atau diperbarui di *cloud* tetapi masih dianggap valid di *edge* karena sinkronisasi tertunda. Dengan demikian, meskipun *edge/fog computing* dan *edge caching* dapat menurunkan latensi, mengurangi permintaan ke sumber data, serta mengurangi ketergantungan pada *cloud*, pendekatan ini belum sepenuhnya menyelesaikan *trade-off* antara kecepatan akses lokal, *freshness attributes*, akurasi keputusan, dan keamanan akses pada lingkungan IoT yang dinamis.

Masalah *freshness attribute* ini bukan sekadar isu teknis yang berdampak pada performa, melainkan juga isu fundamental yang memengaruhi kualitas dan keamanan keputusan otorisasi. Wang dkk. (2025) menunjukkan bahwa mekanisme *edge-based authentication and authorization* perlu mengevaluasi *freshness* nilai atribut dan *trustworthiness* penyedia atribut agar keputusan akses menjadi lebih reliabel. Penelitian tersebut juga menegaskan bahwa *local caching* pada *edge* memang diperlukan ketika

konektivitas *cloud* bersifat terputus-putus, tetapi dapat menimbulkan risiko keamanan karena *stale attribute*. Dalam konteks ini, penggunaan atribut yang usang dapat menyebabkan keputusan otorisasi yang salah, baik berupa *false allow*, yaitu akses tidak sah tetap diizinkan, maupun *false deny*, yaitu akses sah justru ditolak. Risiko *false allow* lebih kritis karena dapat membuka akses tidak sah terhadap sumber daya IoT, terutama ketika *revocation notification* terlambat atau izin yang seharusnya dicabut masih digunakan dalam evaluasi lokal. Oleh karena itu, Wang dkk. (2025) mengusulkan agar *freshness* dan validitas atribut diperiksa sebelum keputusan akses dibuat, serta agar hak akses dikurangi ketika *freshness* atribut lokal menurun, khususnya saat *edge* kehilangan koneksi dengan *cloud*. Di sisi lain, Weerasinghe dkk. (2023) menunjukkan bahwa *caching* pada aplikasi IoT *real-time* selalu melibatkan *trade-off* antara *freshness threshold*, *cache hit*, *response latency*, *retrieval cost*, dan pemenuhan *Service Level Agreement* (SLA). *Time-to-Live* (TTL) yang panjang dapat meningkatkan *cache hit* dan menurunkan latensi, tetapi meningkatkan risiko *stale attribute*. Sebaliknya, TTL yang pendek meningkatkan *freshness*, tetapi memperbanyak pengambilan ulang atribut ke PIP dan menambah latensi. Konsekuensi dari *stale attribute* ini tidak hanya berdampak pada performa, tetapi juga menyebabkan keputusan otorisasi yang salah, sehingga pengelolaan *cache* atribut pada ABAC berbasis *edge* harus diposisikan sebagai masalah keamanan keputusan dan bukan hanya sekadar optimasi performa semata.

Pendekatan TTL statis yang mudah diimplementasikan tidak selalu memadai untuk lingkungan IoT karena data konteks dan atribut akses bersifat dinamis serta memiliki kebutuhan *freshness* yang berbeda-beda. Weerasinghe dkk. (2023) menunjukkan bahwa *adaptive context caching* pada aplikasi IoT *real-time* perlu memertimbangkan *cache hit*, *response latency*, *cost*, *freshness threshold*, dan SLA secara simultan, sehingga strategi *caching* tidak cukup hanya ditentukan oleh aturan statis. Medvedev dkk. (2023) juga menegaskan bahwa *cache management* untuk data IoT menjadi kompleks karena harus memperhitungkan *freshness*, *cost*, latensi, pola permintaan yang tidak pasti, serta variasi SLA yang ada. Dalam konteks otorisasi, kompleksitas ini semakin kritis karena atribut yang berbeda memiliki volatilitas dan dampak keamanan yang berbeda pula. Wang dkk. (2025) menunjukkan bahwa keputusan akses akan menjadi patut dipertanyakan jika sistem mengasumsikan nilai atribut selalu mutakhir dan berasal dari sumber yang sepenuhnya terpercaya. Karena itu, mekanisme *Local Authentication and Authorization* (LAA) perlu memasukkan *freshness* nilai atribut dan *trustworthiness* penyedia atribut untuk meningkatkan reliabilitas keputusan akses. Dengan demikian, konfigurasi TTL, ukuran *cache*, dan pemilihan atribut yang boleh di-*cache* sebaiknya diperlakukan sebagai masalah optimasi adap-

tif, bukan sekadar konfigurasi manual atau heuristik kaku. Hal ini sejalan dengan prinsip *zero-trust* yang menuntut kontrol akses dievaluasi secara kontinu ketika atribut pengguna, perangkat, lokasi, dan perilaku berubah (Azad dkk., 2024).

Algoritma metaheuristik, khususnya *Grey Wolf Optimization* (GWO), menawarkan pendekatan yang menjanjikan untuk menyelesaikan masalah optimasi multi-parameter pada lingkungan IoT, *fog*, dan *edge computing* karena mampu mengeksplorasi ruang solusi yang luas tanpa membutuhkan model analitik tertutup. Alsadie dan Alsulami (2025) mengembangkan TS-GWO untuk *task schedulling* IoT pada lingkungan *fog-cloud* dan menunjukkan peningkatan *makespan* hingga 46,15% serta penurunan konsumsi energi hingga 28,75% dengan keunggulan utama berasal dari kemampuan GWO dalam menyeimbangkan proses eksplorasi dan eksploitasi pada kondisi *fog-cloud* yang dinamis. Temuan serupa ditunjukkan oleh Alsamarai dan Uçan (2024) yang mengusulkan PC-GWO dengan menggabungkan *Performance and Cost Algorithm* dan GWO. Hasilnya menunjukkan peningkatan *resource utilization* hingga 13,2%, penurunan *makespan* hingga 16,27%, dan penurunan konsumsi energi hingga 12,17%. Selain itu, Shreyas dkk. (2025) menunjukkan bahwa *Chaotic Grey Wolf Optimization* (CGWO) mampu mengungguli *Ant Colony Optimization* (ACO) dan *min-max* dalam *task schedulling cloud-fog* berdasarkan *makespan*, *response time*, dan *resource utilization*. Meskipun demikian, penggunaan GWO dalam literatur tersebut masih dominan diarahkan pada *task schedulling*, *resource allocation*, *energy efficiency*, dan *cost optimization*, bukan pada optimasi parameter keamanan ABAC. Padahal, literatur *edge-IAM* terbaru menunjukkan bahwa *freshness* atribut memengaruhi reliabilitas keputusan akses, sementara *local caching* dapat menimbulkan risiko *stale attribute* ketika data yang tersimpan di *edge* tidak lagi sesuai dengan kondisi aktual di *cloud* atau sumber atribut (Wang dkk., 2025). Di sisi lain, *adaptive context caching* pada IoT juga menunjukkan bahwa pengaturan *cache* perlu mempertimbangkan *cost*, latensi, performa, dan *freshness* secara simultan. Dengan demikian, terdapat *research gap* yang jelas untuk menerapkan GWO atau metaheuristik dalam mengoptimalkan TTL atribut, ukuran *cache*, dan strategi *cacheable attribute* pada *edge authorization* berbasis ABAC.

Berdasarkan latar belakang masalah tersebut maka diusulkanlah mekanisme optimasi *cache* atribut dan *freshness* pada ABAC berbasis *edge* yang mempertahankan semantik kebijakan asli. Optimasi hanya menentukan atribut mana yang boleh di-*cache* lokal di *gateway* tanpa melemahkan kebijakan akses. Formulasi optimasi *freshness* ini menggunakan pendekatan metaheuristik yang menyeimbangkan latensi, jumlah *PIP calls*, efisiensi *cache*, dan kualitas keputusan akses. Berbeda dari *caching* biasa, *stale attribute* diperlakukan sebagai risiko keamanan yang diukur melalui *false allow*

rate, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*. Evaluasi akan beracuan pada metrik komprehensif tersebut untuk penerapan ABAC di lingkungan IoT *edge* yang dinamis. Pendekatan ini memadukan optimasi *cache*, manajemen *freshness*, dan kesadaran keamanan secara terpadu.

I.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini merumuskan beberapa masalah utama sebagai berikut:

1. Bagaimana merancang mekanisme ABAC berbasis *edge* pada lingkungan IoT yang mampu menyediakan kontrol akses *fine-grained* dan *context-aware* secara efisien?
2. Bagaimana mekanisme *cache* atribut lokal pada *gateway edge* dapat mengurangi latensi dan *PIP calls* tanpa mengubah semantik kebijakan ABAC?
3. Bagaimana menentukan konfigurasi *cacheable attribute*, TTL per atribut, dan ukuran *cache* yang optimal untuk menjaga keseimbangan antara efisiensi sistem dan *attribute freshness*?
4. Bagaimana algoritma *Grey Wolf Optimization* dapat digunakan untuk mengoptimalkan konfigurasi *cache* atribut dan *freshness* pada ABAC berbasis *edge*?
5. Bagaimana kinerja mekanisme yang diusulkan dibandingkan dengan *cloud-only ABAC*, *edge fresh ABAC*, *static cache*, *adaptive TTL*, dan *random search*?
6. Bagaimana pengaruh mekanisme yang diusulkan terhadap kualitas keputusan otorisasi, khususnya *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate* pada berbagai skenario dinamika *workload*?

I.3 Tujuan Penelitian

Adapun tujuan yang ingin dicapai melalui penelitian ini adalah:

1. Merancang mekanisme ABAC berbasis *edge* untuk mendukung kontrol akses *fine-grained* dan *context-aware* pada lingkungan IoT dinamis.
2. Menerapkan *cache* atribut lokal pada *gateway edge* untuk mengurangi latensi dan jumlah *PIP calls* tanpa menghilangkan atribut dari evaluasi kebijakan ABAC.

3. Memformulasikan optimasi *cache* atribut dan *freshness* melalui variabel *cacheable attribute*, TTL per atribut, dan ukuran *cache*.
4. Menerapkan *Grey Wolf Optimization* untuk mencari konfigurasi *cache* atribut dan *freshness* yang optimal pada ABAC berbasis *edge*.
5. Mengevaluasi performa mekanisme yang diusulkan dibandingkan dengan *cloud-only ABAC*, *edge fresh ABAC*, *static cache*, *adaptive TTL*, dan *random search*.
6. Menganalisis kualitas keputusan otorisasi berdasarkan *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate* pada skenario *workload* rendah, sedang, tinggi, dan sangat tinggi.

I.4 Kontribusi Penelitian

Penelitian ini memberikan kontribusi sebagai berikut:

1. Merancang mekanisme ABAC berbasis *edge* dengan *cache* atribut lokal pada *gateway* untuk mengurangi latensi dan *PIP calls* pada lingkungan IoT dinamis.
2. Mengusulkan strategi *cacheable attribute* yang tetap mempertahankan semantik kebijakan ABAC sehingga optimasi tidak menghapus atribut dari proses evaluasi akses.
3. Memformulasikan pengelolaan *attribute freshness* sebagai masalah optimasi dengan variabel keputusan berupa *cacheable attribute*, TTL per atribut, dan ukuran *cache* lokal.
4. Menerapkan *Grey Wolf Optimization* untuk mencari konfigurasi *cache* dan *freshness* yang mampu menyeimbangkan performa dan kualitas keputusan otorisasi.
5. Mengembangkan fungsi *fitness* yang mempertimbangkan latensi, *PIP calls*, *cache hits*, *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*.
6. Mengevaluasi mekanisme yang diusulkan terhadap beberapa *baseline*, yaitu *cloud-only ABAC*, *edge fresh ABAC*, *static cache ABAC*, *adaptive TTL ABAC*, *random search*, dan *GWO-optimized ABAC*.
7. Menguji mekanisme pada skenario *workload* dinamis dan *threat-aware* dengan mempertimbangkan *stale attribute*, *PIP unavailability*, dan *revocation event*.

I.5 Hipotesis dan Premis

Penelitian ini dibangun di atas premis bahwa penerapan ABAC berbasis *edge* pada lingkungan IoT tidak hanya bergantung pada kemampuan model dalam menyediakan kontrol akses yang *fine-grained* dan *context-aware*, tetapi juga pada cara atribut dikelola selama proses evaluasi akses. Pengambilan atribut secara langsung dari PIP atau *cloud* dapat menjaga kemutakhiran atribut, tetapi meningkatkan latensi dan jumlah *PIP calls*. Sebaliknya, *cache* atribut pada *gateway edge* dapat menurunkan latensi, tetapi berisiko menghasilkan *stale attribute* yang dapat menyebabkan *false allow*, *false deny*, atau keterlambatan pencabutan akses.

Berdasarkan premis tersebut, penelitian ini mengusulkan *GWO-optimized ABAC*, yaitu mekanisme ABAC berbasis *edge* yang menggunakan GWO untuk menentukan konfigurasi *cacheable attribute*, TTL per atribut, dan ukuran *cache* lokal pada *gateway*. Hipotesis penelitian dirumuskan sebagai berikut:

H1 – Efisiensi performa *GWO-optimized ABAC*. *GWO-optimized ABAC* mampu menurunkan latensi keputusan dan jumlah *PIP calls* dibandingkan *static cache ABAC*, *adaptive TTL ABAC*, dan *random search*.

H2 – Kualitas keputusan otorisasi *GWO-optimized ABAC*. *GWO-optimized ABAC* mampu mempertahankan kualitas keputusan otorisasi dengan menjaga *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate* pada tingkat yang lebih terkendali dibandingkan pendekatan *cache* berbasis konfigurasi statis atau manual.

H3 – Keunggulan konfigurasi hasil GWO. Konfigurasi *cacheable attribute*, TTL per atribut, dan ukuran *cache* yang dihasilkan oleh GWO lebih efektif dibandingkan konfigurasi manual dan *random search* dalam menyeimbangkan efisiensi *cache*, *attribute freshness*, dan risiko keputusan akses.

H4 – Ketahanan *GWO-optimized ABAC* terhadap *workload* dinamis. *GWO-optimized ABAC* mampu mempertahankan performa dan kualitas keputusan akses pada skenario *workload* rendah, sedang, tinggi, dan sangat tinggi yang melibatkan perubahan *trust*, *anomaly*, kondisi jaringan, status *firmware*, *role*, *critical request*, *PIP unavailability*, dan *revocation event*.

H5 – Trade-off performa dan keamanan pada *GWO-optimized ABAC*. *GWO-optimized ABAC* mampu menghasilkan *trade-off* yang lebih seimbang antara penurunan latensi dan pengendalian risiko *stale attribute* dibandingkan pendekatan yang hanya menekankan *cache hit* atau TTL statis.

Secara formal, intuisi dasar penelitian ini dapat dinyatakan sebagai berikut. Jika keuntungan performa dari penggunaan *cache* atribut dinyatakan sebagai G , sedangkan risiko keamanan akibat *stale attribute* dinyatakan sebagai R_s , maka konfigurasi *cache* layak digunakan ketika $G > R_s$. Namun karena *false allow* merupakan pelanggaran keamanan yang lebih kritis daripada sekadar peningkatan latensi, tujuan *GWO-optimized ABAC* bukan hanya memaksimalkan *cache hit* atau menurunkan *PIP calls*, tetapi mencari konfigurasi yang mampu mendorong sistem dari kondisi latensi rendah tetapi risiko *stale* tinggi menuju kondisi latensi rendah dengan risiko keputusan akses terkendali. Dengan demikian, target utama penelitian ini adalah *GWO-optimized ABAC* mampu memberikan keseimbangan yang lebih baik antara efisiensi evaluasi akses, *attribute freshness*, dan keamanan keputusan otorisasi dibandingkan pendekatan ABAC berbasis *edge* dengan konfigurasi manual atau *non-optimized*.

I.6 Batasan Masalah

Untuk menjaga fokus dan kedalaman, penelitian ini dibatasi oleh beberapa hal berikut:

1. Penelitian difokuskan pada aspek otorisasi dalam sistem kontrol akses IoT, sehingga tidak membahas autentikasi, manajemen identitas penuh, enkripsi komunikasi, maupun integrasi dengan *identity provider* eksternal.
2. Model kontrol akses yang digunakan dibatasi pada ABAC. Model lain seperti RBAC, DAC, MAC, UCON, atau CapBAC hanya dibahas sebagai pembanding konseptual.
3. Arsitektur penelitian dibatasi pada ABAC berbasis *edge*, dengan *gateway* sebagai tempat evaluasi keputusan akses dan pengelolaan *cache* atribut.
4. Parameter yang dioptimasi dibatasi pada *cacheable attribute*, TTL per atribut, dan ukuran *cache* lokal pada *gateway*. Penelitian ini tidak mengoptimalkan struktur kebijakan ABAC secara penuh.
5. Optimasi yang dilakukan tidak mengubah semantik kebijakan ABAC. Seluruh atribut tetap dievaluasi dalam proses pengambilan keputusan akses, sedangkan optimasi hanya menentukan atribut mana yang diperbolehkan untuk di-*cache*.
6. Atribut yang digunakan dalam eksperimen dibatasi pada atribut simulatif, yaitu *trust*, *anomaly*, *network condition*, *role*, *firmware status*, *critical request*, *PIP availability*, dan *revocation event*.

7. Pengujian dilakukan menggunakan *workload* sintetis dengan empat skenario dinamika, yaitu *low dynamic*, *medium dynamic*, *high dynamic*, dan *very dynamic*. Penelitian ini tidak menggunakan trafik IoT nyata.
8. Lingkungan eksperimen dilakukan melalui simulasi berbasis iFogSim2 dan Java, bukan *deployment* langsung pada perangkat IoT fisik.
9. Metrik evaluasi performa dibatasi pada *average latency*, *p95 latency*, *PIP calls*, *cache hits*, *CPU time*, dan *memory usage*.
10. Metrik kualitas keputusan akses dibatasi pada *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*.
11. Algoritma yang dikaji adalah GWO. Algoritma lain seperti *random search* digunakan sebagai pembandingan, bukan fokus utama penelitian.
12. *Threat model* penelitian ini dibatasi pada risiko *stale attribute*, *revocation delay*, dan *PIP unavailability*. Serangan seperti *physical tampering*, *side-channel*, *malware gateway*, *man-in-the-middle*, dan kompromi penuh terhadap *cloud* tidak dibahas secara mendalam.
13. Hasil penelitian bersifat evaluatif dalam konteks simulasi, sehingga validasi pada perangkat fisik atau *testbed* IoT nyata menjadi bagian dari pengembangan penelitian selanjutnya.

I.7 Metodologi Penelitian

Penelitian ini menggunakan pendekatan *Design Science Research* (DSR) yang diperkuat dengan *simulation-based experimental evaluation*. Pendekatan DSR digunakan karena penelitian ini tidak hanya menganalisis fenomena keamanan pada ABAC berbasis *edge*, tetapi juga merancang artefak berupa mekanisme *GWO-optimized ABAC* untuk mengoptimalkan *cache* atribut dan *attribute freshness* pada lingkungan IoT dinamis.

Pada tahap pertama, penelitian melakukan identifikasi masalah dan motivasi berdasarkan kajian literatur mengenai kontrol akses IoT, ABAC, *edge/fog computing*, *attribute freshness*, *stale attribute*, dan optimasi metaheuristik. Tahap ini menegaskan bahwa masalah utama dalam penerapan ABAC pada IoT tidak hanya terletak pada kemampuan model dalam menyediakan kontrol akses yang *fine-grained* dan *context-aware*, tetapi juga pada biaya pengambilan atribut mutakhir dari PIP atau *cloud* serta risiko penggunaan atribut usang ketika atribut disimpan secara lokal di *edge*.

Pada tahap kedua, penelitian memformalkan masalah ke dalam model konseptual. Variabel konteks yang digunakan meliputi dinamika *workload*, perubahan nilai atribut, status konektivitas PIP, dan kejadian *revocation*. Variabel keputusan meliputi *cacheable attribute*, nilai TTL per atribut, dan ukuran *cache* lokal pada *gateway edge*. variabel *outcome* meliputi *average latency*, *p95 latency*, jumlah *PIP calls*, *cache hits*, *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*.

Pada tahap ketiga, penelitian merancang mekanisme *GWO-optimized ABAC*. Dalam mekanisme ini, seluruh atribut tetap digunakan dalam proses evaluasi kebijakan sehingga semantik kebijakan tidak diubah. Optimasi hanya dilakukan pada strategi pengelolaan atribut, yaitu menentukan atribut mana yang boleh di-*cache*, berapa nilai TTL masing-masing atribut, dan berapa ukuran *cache* lokal pada *gateway*. Fungsi *fitness* dirancang untuk menyeimbangkan efisiensi performa dan kualitas keputusan otorisasi dengan memberi penalti terhadap risiko keamanan seperti *false allow*, *false deny*, dan penggunaan *stale attribute*.

Pada tahap keempat, penelitian mengimplementasikan model eksperimen menggunakan Java dan representasi topologi *edge/fog* berbasis iFogSim2. Topologi yang digunakan terdiri dari *cloud*, *proxy*, *gateway edge*, dan *endpoint IoT*. *Gateway edge* berperan sebagai lokasi evaluasi keputusan akses dan pengelolaan *cache* atribut. *Workload* dibangkitkan secara sintesis dengan empat tingkat dinamika, yaitu:

1. *Low dynamic workload*;
2. *Medium dynamic workload*;
3. *High dynamic workload*; dan
4. *Very dynamic workload*.

Pada tahap kelima, penelitian menjalankan eksperimen komparatif terhadap beberapa konfigurasi, yaitu:

1. *cloud-only ABAC*, yaitu evaluasi akses bergantung pada atribut dari *cloud*/PIP pusat;
2. *edge fresh ABAC*, yaitu evaluasi dilakukan di *edge* tetapi atribut selalu diambil secara mutakhir;
3. *static cache ABAC*, yaitu *cache* atribut menggunakan TTL tetap;
4. *adaptive TTL ABAC*, yaitu TTL disesuaikan menggunakan aturan heuristik manual;

5. *random search ABAC*, yaitu konfigurasi *cache* dicari secara acak sebagai *baseline* optimasi;
6. *GWO-optimized ABAC*, yaitu konfigurasi *cache* dioptimasi menggunakan GWO sebagai model yang diusulkan.

Pada tahap keenam, penelitian membagi *workload* ke dalam data optimasi dan data pengujian. Data optimasi digunakan oleh GWO dan *random search* untuk mencari konfigurasi *cacheable attribute*, TTL per atribut, dan ukuran *cache*. Data pengujian digunakan untuk mengevaluasi performa akhir setiap metode agar hasil tidak bias terhadap data yang digunakan selama proses optimasi.

Pada tahap ketujuh, hasil eksperimen dianalisis untuk menjawab apakah *GWO-optimized ABAC* mampu memberikan keseimbangan yang lebih baik antara efisiensi evaluasi akses, *attribute freshness*, dan kualitas keputusan otorisasi dibandingkan pendekatan lain. Analisis dilakukan dengan membandingkan metrik performa dan keamanan antar skenario, khususnya *average latency*, *p95 latency*, *PIP calls*, *cache hits*, *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*. Dengan demikian, evaluasi tidak hanya menilai apakah mekanisme yang diusulkan lebih cepat, tetapi juga apakah keputusan akses yang dihasilkan tetap aman dan reliabel pada lingkungan IoT *edge* yang dinamis.

I.8 Sistematika Penulisan

Tesis ini disusun dalam lima bab utama.

Bab I memuat pendahuluan yang mencakup latar belakang masalah, rumusan masalah, tujuan penelitian, kontribusi penelitian, hipotesis dan premis, batasan penelitian, metodologi penelitian, serta sistematika penulisan.

Bab II memuat tinjauan pustaka dan landasan teori mengenai IoT, keamanan dan kontrol akses pada IoT, ABAC, arsitektur *edge/fog computing*, *attribute freshness*, mekanisme *cache* atribut, TTL, algoritma metaheuristik, serta posisi penelitian ini di antara studi-studi lainnya.

Bab III memuat metodologi penelitian yang menjelaskan model konseptual, arsitektur ABAC berbasis *edge*, *threat model*, desain mekanisme *cache* atribut, formulasi optimasi *cacheable attribute*, TTL, dan ukuran *cache*, desain algoritma *GWO-optimized ABAC*, skenario *workload*, *baseline* eksperimen, metrik evaluasi, serta prosedur pengujian.

Bab IV memuat implementasi, pengujian, analisis, dan pembahasan, termasuk detail

implementasi simulasi berbasis iFogSim2 dan Java, konfigurasi lingkungan eksperimen, hasil pengujian pada berbagai skenario dinamika, serta perbandingan kinerja antar konfigurasi uji. Bab ini juga membahas *trade-off* antara latensi, *PIP calls*, *cache efficiency*, *attribute freshness*, *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*.

Bab V memuat kesimpulan dan saran untuk penelitian lanjutan. Bab ini merangkum temuan utama penelitian, menilai pencapaian tujuan penelitian, menjelaskan kontribusi *GWO-optimized ABAC* terhadap efisiensi evaluasi akses dan kualitas keputusan otorisasi, serta memberikan arah pengembangan lebih lanjut, seperti pengujian pada *testbed* IoT fisik, integrasi dengan mekanisme autentikasi nyata, dan implementasi *event-based simulation* yang lebih lengkap pada iFogSim2.

BAB II TINJAUAN PUSTAKA DAN LANDASAN TEORI

II.1 IoT, Keamanan, dan Kebutuhan Kontrol Akses

Internet of Things (IoT) merupakan ekosistem yang menghubungkan berbagai perangkat fisik, sensor, aplikasi, dan layanan melalui jaringan komunikasi untuk mendukung pertukaran data serta otomatisasi layanan secara *real-time*. Lingkungan IoT bersifat heterogen karena terdiri dari perangkat dengan kemampuan, protokol, dan sistem operasi yang berbeda-beda, serta terdistribusi karena perangkat tersebar pada berbagai lokasi dan saling berkomunikasi melalui jaringan yang luas. Selain itu, sebagian besar perangkat IoT memiliki keterbatasan sumber daya pada aspek komputasi, memori, energi, dan *bandwidth*, sehingga mekanisme keamanan yang digunakan harus ringan dan efisien. Karakteristik IoT yang *latency-sensitive* juga menuntut respons yang cepat karena banyak aplikasi IoT digunakan pada lingkungan kritis seperti kesehatan, industri, dan *smart transportation* (Kokila dan Reddy K, 2025).

Dari sisi keamanan, lingkungan IoT memiliki *attack surface* yang sangat luas akibat jumlah perangkat yang besar dan konektivitas yang terus meningkat. Williams dkk. (2022) menjelaskan bahwa ancaman keamanan IoT dapat muncul pada berbagai lapisan, mulai dari perangkat, jaringan, hingga aplikasi. Kerentanan tersebut semakin diperparah oleh penggunaan *firmware* yang tidak aman, mekanisme autentikasi yang lemah, serta pengelolaan *trust* yang tidak memadai, sehingga meningkatkan risiko akses tidak sah terhadap perangkat dan data IoT (Altaibek dkk., 2025). Selain itu, konektivitas jaringan IoT yang sering berubah dan tidak stabil membuat proses pengamanan akses menjadi lebih kompleks karena kondisi perangkat dan konteks lingkungan dapat berubah secara dinamis (Krishnasrija dkk., 2026).

Dalam konteks keeamanan IoT, autentikasi dan otorisasi memiliki fungsi yang berbeda tetapi saling berkaitan. Autentikasi bertujuan memverifikasi identitas pengguna atau perangkat, sedangkan otorisasi menentukan tindakan apa saja yang diperbolehkan setelah identitas tersebut berhasil diverifikasi (Díaz dan Mendoza, 2025). Pada lingkungan IoT yang dinamis, keputusan otorisasi tidak cukup hanya didasarkan

pada identitas atau peran statis, tetapi juga perlu mempertimbangkan atribut kontekstual seperti lokasi, waktu, jenis perangkat, status keamanan perangkat, dan jenis aksi yang diminta. Oleh karena itu, IoT membutuhkan mekanisme kontrol akses yang *fine-grained*, *context-aware*, *scalable*, *low-latency*, dan mampu beradaptasi terhadap perubahan konteks secara *real-time*. Kalaria dkk. (2024) menunjukkan bahwa *adaptive context-aware access control* diperlukan agar kebijakan akses dapat menyesuaikan kondisi lingkungan secara dinamis sekaligus mengurangi ketergantungan pada *cloud* melalui pemrosesan di *fog* atau *edge*. Sementara itu, Wang dkk. (2025) menegaskan bahwa layanan IAM pada IoT masa depan harus mampu menyediakan *access management* yang aman, skalabel, dan berlatensi rendah agar sesuai dengan kebutuhan layanan IoT.

II.2 Model Kontrol Akses dan Relevansi ABAC pada IoT

Model kontrol akses digunakan untuk menentukan bagaimana suatu sistem mengatur hak akses pengguna atau perangkat terhadap sumber daya tertentu. Salah satu model paling awal adalah *Discretionary Access Control* (DAC), yaitu model yang memberikan kewenangan kepada pemilik sumber daya untuk memberikan atau menolak izin akses kepada identitas atau kelompok tertentu melalui mekanisme seperti *Access Control List* (ACL) (Díaz dan Mendoza, 2025). Model ini relatif fleksibel karena pengelolaan izin dilakukan langsung oleh pemilik objek, namun pendekatan tersebut menyulitkan penerapan kebijakan keamanan yang terpusat dan konsisten pada lingkungan berskala besar. Mao dkk. (2025) menyatakan bahwa DAC menawarkan fleksibilitas tetapi terlalu sulit untuk menerapkan kebijakan keamanan yang terpadu, sehingga dapat menyebabkan *permission creep* dan inkonsistensi kebijakan keamanan.

Sementara itu, *Mandatory Access Control* (MAC) menggunakan pendekatan berbasis label atau level keamanan yang ditentukan secara terpusat oleh administrator sistem. Dalam model ini, hak akses ditentukan berdasarkan perbandingan atribut keamanan pengguna dan objek terhadap level otorisasi yang telah ditetapkan (Díaz dan Mendoza, 2025). MAC menawarkan tingkat keamanan yang tinggi karena kebijakan akses dikendalikan secara ketat oleh sistem, tetapi model ini bersifat *rigid* dan kurang adaptif terhadap perubahan kebutuhan akses yang dinamis. Mao dkk. (2025) menjelaskan bahwa konfigurasi kebijakan MAC terlalu *rigid* dan sulit menyesuaikan diri dengan kebutuhan akses yang sering berubah pada lingkungan *cloud* yang dinamis dan kompleks. Kondisi tersebut membuat MAC kurang sesuai diterapkan pada lingkungan IoT yang memiliki karakteristik heterogen, dinamis, dan membutuhkan fleksibilitas

tinggi dalam pengelolaan akses. Ragothaman dkk. (2023) juga menekankan bahwa lingkungan IoT memiliki kebutuhan akses kontrol yang bersifat dinamis, heterogen, dan membutuhkan granularitas tinggi dalam pengambilan keputusan akses.

Selain DAC dan MAC, *Role-Based Access Control* (RBAC) dikembangkan untuk menyederhanakan pengelolaan hak akses melalui konsep peran (*role*). Dalam RBAC, pengguna tidak diberikan izin secara langsung, melainkan memperoleh izin akses melalui peran yang dimiliki. Ferraiolo dkk. (2001) menjelaskan bahwa konsep dasar RBAC adalah pengguna dikelompokkan berdasarkan peran, izin diberikan kepada peran, dan pengguna memperoleh izin dengan menjadi anggota suatu peran. Pendekatan ini mempermudah administrasi hak akses karena pengelolaan izin akses dapat dilakukan pada level peran, bukan pada setiap pengguna secara individual. RBAC juga banyak digunakan pada sistem berskala besar karena mampu mendukung pengelolaan hak akses yang lebih terstruktur dan efisien (Ferraiolo dkk., 2001). Namun, pada lingkungan IoT yang sangat dinamis dan heterogen, RBAC memiliki keterbatasan dalam menangani kebutuhan akses yang bersifat kontekstual. Díaz dan Mendoza (2025) menyatakan bahwa RBAC dapat mengalami kesulitan pada lingkungan IoT karena sering membutuhkan tambahan faktor seperti kondisi lingkungan, waktu, dan lokasi dalam proses pengambilan keputusan akses. Selain itu, Ameer dkk. (2023) menegaskan bahwa pada lingkungan *cloud* dan IoT yang kompleks, RBAC dapat mengalami masalah *role explosion* ketika jumlah peran dan kombinasi izin akses meningkat secara signifikan.

Untuk mengatasi keterbatasan model akses tradisional yang hanya melakukan evaluasi sebelum akses diberikan, dikembangkan konsep *Usage Control* (UCON) yang mendukung kontrol akses yang lebih dinamis dan berkelanjutan. Park dan Sandhu (2004) menjelaskan bahwa UCON merupakan generalisasi dari kontrol akses yang mencakup otorisasi, obligasi, kondisi, kontinuitas, dan mutabilitas. Berbeda dengan model tradisional yang umumnya hanya melakukan keputusan akses pada saat permintaan awal, UCON juga mendukung *ongoing control* selama sesi penggunaan berlangsung. Dengan demikian, hak akses dapat berubah secara dinamis berdasarkan perubahan atribut, kondisi lingkungan, maupun perilaku pengguna selama sesi aktif. UCON juga mendukung pembaruan atribut secara terus-menerus sebagai konsekuensi dari aktivitas akses yang sedang berlangsung (Park dan Sandhu, 2004). Meskipun menawarkan fleksibilitas dan kontrol yang lebih kaya dibandingkan DAC, MAC, dan RBAC, implementasi UCON cenderung lebih kompleks karena membutuhkan pemantauan atribut, kondisi, dan evaluasi kebijakan secara kontinu selama proses penggunaan sumber daya berlangsung.

Attribute-Based Access Control (ABAC) menjadi salah satu pendekatan yang paling relevan untuk IoT karena keputusan akses dibuat berdasarkan atribut subjek, objek, aksi, dan lingkungan yang berkaitan dengan permintaan akses (Hu dkk., 2014). Dengan pendekatan ini, keputusan akses tidak hanya ditentukan oleh identitas atau peran, tetapi juga oleh konteks seperti lokasi, waktu, jenis perangkat, status keamanan perangkat, dan jenis aksi yang diminta. Díaz dan Mendoza (2025) menjelaskan bahwa ABAC mampu menyediakan *fine-grained access control* karena izin dapat ditentukan secara lebih spesifik berdasarkan kombinasi atribut yang relevan. Selain itu, Kaloria dkk. (2024) menunjukkan bahwa kontrol akses IoT perlu bersifat adaptif dan *context-aware* agar mampu menyesuaikan kebijakan akses terhadap perubahan kondisi lingkungan secara dinamis. Karakteristik tersebut membuat ABAC lebih sesuai untuk IoT dibandingkan DAC, MAC, maupun RBAC yang cenderung lebih statis.

Meskipun demikian, penerapan ABAC pada IoT juga memiliki tantangan tersendiri. ABAC membutuhkan banyak atribut dari berbagai sumber untuk mengevaluasi setiap permintaan akses, sehingga proses pengambilan atribut dari PIP, *cloud*, atau penyedia atribut lain dapat meningkatkan *overhead* komunikasi dan latensi. Cremonezi dkk. (2022) menunjukkan bahwa *attribute retrieval* pada ABAC di lingkungan *fog-IoT* dapat menjadi mahal pada skenario IoT yang besar dan *mobile*, sehingga strategi *caching* atribut diperlukan untuk meningkatkan performa. Selain itu, kualitas keputusan akses sangat bergantung pada *freshness* atribut yang digunakan. Wang dkk. (2025) menegaskan bahwa keputusan akses pada *edge-IAM* dapat menjadi tidak reliabel apabila sistem mengasumsikan atribut selalu mutakhir meskipun kondisi aktual telah berubah. Oleh karena itu, meskipun ABAC paling sesuai untuk kebutuhan kontrol akses IoT yang dinamis, penerapannya tetap membutuhkan strategi pengelolaan atribut yang efisien agar performa dan keamanan sistem dapat terjaga secara seimbang.

II.3 Dasar Pemilihan Atribut Kebijakan ABAC

Pemilihan atribut kebijakan ABAC dalam penelitian ini didasarkan pada kebutuhan untuk merepresentasikan keputusan akses IoT secara lebih komprehensif, bukan hanya berdasarkan identitas subjek. Secara konseptual, ABAC menentukan keputusan akses berdasarkan atribut subjek, objek, aksi, dan lingkungan, sehingga atribut yang dipilih perlu mencakup identitas pemohon, karakteristik sumber daya, jenis aksi, serta kondisi lingkungan akses (Hu dkk., 2014). Oleh karena itu, atribut *role* digunakan sebagai representasi *privilege* dasar subjek, tetapi tidak dijadikan satu-satunya dasar

keputusan agar model tetap mencerminkan karakteristik ABAC yang *fine-grained* dan *context-aware*. Díaz dan Mendoza (2025) menegaskan bahwa otorisasi pada IoT menentukan aksi spesifik yang boleh dilakukan setelah autentikasi, sehingga keputusan akses perlu mempertimbangkan atribut yang lebih luas daripada sekadar identitas atau peran.

Kelompok atribut *trust* dan *anomaly* dipilih untuk merepresentasikan tingkat kepercayaan serta indikasi perilaku abnormal pada perangkat atau pengguna. Waheed dkk. (2025) menunjukkan bahwa *dynamic* ABAC pada lingkungan *smart energy* berbasis IoT memanfaatkan mekanisme *trust recalibration* dan analisis perilaku untuk menyesuaikan keputusan akses secara adaptif dan kontekstual. Sementara itu, Mao dkk. (2025) mengintegrasikan *dynamic trust evaluation*, *anomaly traffic detection*, dan *abnormal log detection* sebagai bagian dari perhitungan nilai risiko dalam model *zero-trust access control* berbasis atribut. Dengan demikian, atribut *trust* dan *anomaly* relevan digunakan karena keduanya bersifat dinamis dan mampu merepresentasikan perubahan perilaku aktual entitas selama proses akses berlangsung.

Atribut *network* dan *pipAvailable* dipilih untuk merepresentasikan kondisi lingkungan jaringan serta ketersediaan sumber atribut dalam proses evaluasi kebijakan. Kalaria dkk. (2024) menunjukkan bahwa *adaptive context-aware access control* pada IoT perlu mempertimbangkan konteks yang berubah, termasuk *network context* seperti kondisi jaringan, trafik jaringan, dan komunikasi antar-*node*, agar keputusan akses dapat disesuaikan secara dinamis. Dalam konteks *edge-IAM*, Wang dkk. (2025) menunjukkan bahwa LAA diperlukan untuk mendukung akses berlatensi rendah dan tetap berjalan ketika konektivitas *edge-cloud* tidak selalu tersedia dengan memanfaatkan data IAM lokal, *attribtue freshness*, dan sinkronisasi berbasis konteks. Oleh karena itu, *pipAvailable* dapat digunakan sebagai abstraksi eksperimental untuk menguji kondisi ketika sumber atribut atau layanan informasi kebijakan tidak tersedia, sedangkan *network* merepresentasikankondisi konektivitas yang memengaruhi kemampuan sistem memperoleh atribut terbaru.

Atribut *firmware* digunakan untuk merepresentasikan postur keamanan perangkat. Bakhshi dkk. (2024) menunjukkan bahwa *firmware* pada perangkat IoT merupakan komponen kritis yang sering terabaikan, karena kerentanan *firmware* dapat menjadi target serangan dan perangkat yang tidak dapat diperbarui akan tetap rentan terhadap kelemahan yang ditemukan setelah perangkat dirilis. Selain itu, aspek *image management*, *storage integrity*, dan *update delivery* juga berpengaruh terhadap integritas *firmware*, terutama ketika perangkat menjalankan *firmware* usang atau proses

pembaruannya tidak aman. Altaibek dkk. (2025) juga menegaskan bahwa *firmware backdoor*, *command-and-control* tersembunyi, dan manipulasi sensor merupakan *attack vector* yang dapat merusak integritas *deployment* IoT. Dengan demikian, meskipun *firmware* relatif lebih stabil dibanding atribut perilaku seperti *trust* atau *anomaly*, status *firmware* tetap relevan digunakan sebagai atribut keamanan perangkat karena *firmware* yang usang, tidak aman, atau tidak tervalidasi dapat meningkatkan risiko kompromi dan seharusnya memengaruhi keputusan akses.

Atribut *objectType*, *action*, dan *sensitivity* dipilih agar model kebijakan merepresentasikan ABAC secara utuh, bukan sekadar *subject-based access control*. NIST SP 800-162 menjelaskan bahwa otorisasi dalam ABAC ditentukan dengan mengevaluasi atribut yang terkait dengan subjek, objek, operasi yang diminta, dan dalam beberapa kasus kondisi lingkungan terhadap kebijakan atau aturan akses (Hu dkk., 2014). Ullah dkk. (2023) juga menjelaskan bahwa ABAC mendukung kontrol akses yang dinamis, *fine-grained*, dan *context-aware* dengan mempertimbangkan atribut subjek, sumber daya/objek, serta kondisi lingkungan dalam lingkungan IoT. Oleh karena itu, *objectType* digunakan untuk membedakan jenis sumber daya, *action* untuk merepresentasikan operasi akses seperti *read*, *write*, atau *control*, sedangkan *sensitivity* digunakan sebagai atribut objek yang menunjukkan tingkat kritikalitas atau risiko sumber daya yang diakses.

Atribut *critical* dan *revoked* dipilih untuk mendukung evaluasi kebijakan yang peka terhadap risiko, terutama pada skenario akses berdampak tinggi, pencabutan hak akses, dan potensi *false allow* akibat *stale attribute*. Wang dkk. (2025) menunjukkan bahwa keputusan akses pada *edge-IAM* dapat menjadi tidak andal apabila sistem mengasumsikan nilai atribut selalu mutakhir, karena data yang telah diperbarui atau dicabut di *cloud* dapat tetap digunakan secara lokal di *edge* akibat keterlambatan atau kegagalan sinkronisasi. Azad dkk. (2024) juga menekankan bahwa *zero-trust* membutuhkan verifikasi dan evaluasi akses secara kontinu berdasarkan faktor seperti identitas, kondisi perangkat, lokasi, perilaku, dan prinsip *least-privilege*. Dengan demikian, *revoked* relevan untuk menguji risiko *revocation delay* dan penggunaan atribut yang sudah tidak valid, sedangkan *critical* digunakan sebagai abstraksi eksperimental untuk membedakan permintaan akses berdampak tinggi yang memerlukan evaluasi lebih ketat.

Berdasarkan karakteristik tersebut, setiap atribut memiliki tingkat volatilitas dan dampak keamanan yang berbeda. Atribut seperti *trust*, *anomaly*, dan *network* cenderung berubah cepat sehingga membutuhkan *freshness* yang lebih tinggi. Sebaliknya, atribut seperti *role* dan *firmware* relatif lebih stabil, tetapi dapat menimbulkan

risiko keamanan serius apabila nilai yang usang tetap digunakan dalam keputusan akses. Perbedaan ini menunjukkan bahwa strategi TTL dan *cache* atribut tidak seharusnya diterapkan secara seragam, melainkan perlu disesuaikan dengan volatilitas atribut, sensitivitas sumber daya, serta konsekuensi keamanan dari keputusan akses yang salah.

II.4 Edge/Fog Computing, Attribute Caching, dan Freshness

Edge/fog computing menjadi pendekatan penting dalam arsitektur IoT karena memindahkan sebagian proses komputasi dan pengambilan keputusan lebih dekat ke *endpoint*. Dalam konteks kontrol akses, Kaloria dkk. (2024) menunjukkan bahwa kontrol akses berbasis *fog* dapat membawa *decision-making* dan informasi kebijakan lebih dekat ke *end nodes* sehingga kebijakan akses dapat dievaluasi secara lebih adaptif dan *real-time*. Pendekatan ini juga mengurangi ketergantungan penuh pada *cloud*, terutama ketika aplikasi IoT membutuhkan respons cepat dan konektivitas jaringan tidak selalu stabil. Wang dkk. (2025) juga menekankan pentingnya autentikasi dan otorisasi berbasis *edge* untuk mendukung latensi rendah dan manajemen akses luring pada layanan IoT berbasis *edge*. Pada arsitektur seperti ini, *gateway* atau *fog node* dapat menjadi lokasi strategis untuk menempatkan komponen evaluasi dan penegakan kebijakan, seperti *Policy Decision Point* (PDP) atau *Policy Enforcement Point* (PEP) agar proses otorisasi tidak selalu dilakukan secara terpusat di *cloud*.

Untuk meningkatkan responsivitas, atribut atau konteks yang sering digunakan dapat disimpan sementara pada *edge* melalui mekanisme *caching*. Weerasinghe dkk. (2023) menunjukkan bahwa *adaptive context caching* pada aplikasi IoT dapat meningkatkan efisiensi performa dan biaya ketika sistem harus merespons *context queries* secara *real-time*. Dalam konteks implementasi ABAC/XACML, *caching* atribut dapat mengurangi kebutuhan pengambilan atribut berulang dari sumber informasi kebijakan, memperpendek waktu *retrieval* atribut, dan menurunkan latensi evaluasi kebijakan. Namun, manfaat performa ini bergantung pada kemampuan sistem menjaga agar atribut yang tersimpan di *cache* tetap cukup mutakhir untuk digunakan dalam keputusan akses.

Freshness menjadi isu utama karena atribut yang tersimpan di *cache* hanya valid dalam periode tertentu. Medvedev dkk. (2023) menjelaskan bahwa *freshness*, *accuracy*, *completeness*, dan latensi merupakan bagian dari *Quality of Service* (QoS) dalam pengelolaan konteks IoT sehingga strategi *caching* perlu memperhitungkan *trade-off* antara kualitas informasi dan biaya pengambilan data. Dalam praktik *caching*, periode *refresh* atau TTL yang lebih panjang dapat mengurangi frekuensi

retrieval atribut dan *overhead* komunikasi sehingga meningkatkan efisiensi sistem. Sebaliknya, TTL yang lebih pendek dapat menjaga *freshness* atribut karena data lebih sering diperbarui, tetapi meningkatkan frekuensi *retrieval*, sinkronisasi, dan *overhead* komunikasi yang dapat memengaruhi latensi evaluasi akses.

Masalah *stale attribute* tidak hanya berdampak pada performa, tetapi juga pada kualitas dan keamanan keputusan otorisasi. Wang dkk. (2025) menegaskan bahwa keputusan akses pada *edge-IAM* menjadi dipertanyakan apabila sistem mengasumsikan nilai atribut selalu mutakhir dan berasal dari sumber yang sepenuhnya terpercaya. Penelitian tersebut juga mengusulkan LAA yang memasukkan *freshness* nilai atribut dan *trustworthiness* penyedia atribut agar kontrol akses menjadi lebih reliabel. Dengan demikian, atribut yang sudah usang dapat menyebabkan keputusan yang salah, baik berupa *false allow* ketika akses yang seharusnya ditolak justru diizinkan, *false deny* ketika akses sah ditolak, maupun *revocation false allow* ketika hak akses yang sudah dicabut masih dianggap valid di *edge*.

Oleh karena itu, *caching* atribut pada *edge* perlu diperlakukan sebagai persoalan optimasi keamanan, bukan sekadar optimasi performa. Strategi TTL yang seragam untuk semua atribut kurang memadai karena setiap atribut memiliki tingkat volatilitas dan dampak keamanan yang berbeda. Atribut yang sangat dinamis atau berdampak tinggi terhadap keamanan membutuhkan *refresh* yang lebih ketat, sedangkan atribut yang lebih stabil dapat diberi TTL yang lebih panjang untuk mengurangi beban *retrieval* atribut. Dengan demikian, konfigurasi TTL per atribut, ukuran *cache*, dan pemilihan *cacheable attribute* perlu dioptimalkan agar sistem dapat menjaga keseimbangan antara otorisasi berlatensi rendah, efisiensi komunikasi, *freshness* atribut, dan kualitas keputusan akses.

II.5 Threat Model dan Metrik Evaluasi Keamanan

Threat model dalam evaluasi ABAC berbasis *edge* perlu mencakup ancaman yang muncul akibat penggunaan atribut lokal, keterbatasan konektivitas, serta dinamika sinkronisasi atribut dan keputusan otorisasi. NIST SP 800-162 menjelaskan bahwa keputusan akses dalam ABAC sangat bergantung pada validitas, *freshness*, dan kepercayaan terhadap atribut yang digunakan dalam evaluasi kebijakan (Hu dkk., 2014). NIST juga menekankan bahwa perubahan atribut atau *privilege* dapat memengaruhi keputusan akses secara dinamis tanpa perlu mengubah relasi subjek-objek secara langsung. Dalam konteks *edge-IAM*, Wang dkk. (2025) menegaskan bahwa keputusan akses menjadi dipertanyakan apabila sistem mengasumsikan nilai atribut selalu mutakhir dan berasal dari sumber yang sepenuhnya terpercaya. Selain itu,

Wang dkk. (2025) juga menjelaskan bahwa *local caching* dan sinkronisasi atribut pada *edge* dapat menimbulkan risiko penggunaan *stale attribute* akibat pembaruan atribut yang tertunda atau hilang. Oleh karena itu, *threat model* penelitian ini mencakup *stale attribute*, *stale role*, *revocation delay*, *PIP unavailable*, *false allow*, *false deny*, serta *revocation false allow* sebagai ancaman yang relevan pada implementasi ABAC/XACML terdistribusi di lingkungan *edge* IoT.

False allow diposisikan sebagai pelanggaran keamanan karena sistem mengizinkan akses yang seharusnya ditolak. Sebaliknya, *false deny* lebih berkaitan dengan *availability* atau *usability* karena sistem menolak akses yang sebenarnya sah. *Revocation false allow* menjadi kasus khusus yang lebih kritis karena hak akses yang telah dicabut masih dapat digunakan akibat keterlambatan sinkronisasi atau penggunaan atribut *cache* yang sudah usang. Ancaman ini konsisten dengan penjelasan Wang dkk. (2025) mengenai risiko penggunaan atribut usang dalam LAA pada *edge-IAM*. Dengan demikian, *stale cache hit* tidak hanya menjadi indikator performa *cache*, tetapi juga indikator risiko *freshness* yang dapat memengaruhi kualitas dan reliabilitas keputusan otorisasi.

Ancaman *PIP unavailable* digunakan untuk merepresentasikan kondisi ketika sistem tidak dapat memperoleh atribut terbaru dari sumber atribut. Dalam arsitektur ABAC, PDP membutuhkan atribut dan informasi kebijakan dari berbagai komponen untuk melakukan evaluasi akses. Tall dan Zou (2023) menjelaskan bahwa keputusan akses dalam ABAC bergantung pada *trust chain* yang melibatkan atribut subjek, objek, lingkungan, dan mekanisme evaluasi kebijakan. Dalam konteks *edge-IAM*, Wang dkk. (2025) menunjukkan bahwa LAA pada *edge* bergantung pada sinkronisasi atribut dari *cloud* dan dapat menghadapi risiko *stale attribute* ketika konektivitas atau sinkronisasi terganggu. Oleh karena itu, ketika sumber atribut atau PIP tidak tersedia, sistem *edge* harus memilih antara menggunakan atribut lokal yang berpotensi usang atau menolak akses karena informasi tidak cukup untuk membuat keputusan yang reliabel.

Selain itu, *cache poisoning* dan *time manipulation* terhadap TTL dapat diposisikan sebagai ancaman konseptual atau batasan lanjutan. Yu dkk. (2025) menunjukkan bahwa *cache poisoning* terjadi ketika isi *cache* tidak lagi konsisten dengan sumber otoritatif akibat manipulasi respons *cache*. Sementara itu, Losty dkk. (2025) menunjukkan bahwa kegagalan mengikuti TTL dan manipulasi TTL pada sistem DNS IoT dapat memicu ketidakkonsistenan *query* dan meningkatkan risiko *cache poisoning*. Meskipun ancaman tersebut dibahas pada konteks *DNS caching*, konsep yang sama relevan secara konseptual pada *caching* atribut ABAC karena perubahan TTL dapat

memengaruhi *freshness* atribut dan konsistensi keputusan otorisasi.

Evaluasi ABAC berbasis *edge* tidak cukup hanya menggunakan metrik performa seperti *average latency* dan *p95 latency*. Weerasinghe dkk. (2023) menunjukkan bahwa evaluasi *caching* IoT perlu mempertimbangkan latensi, kemungkinan *delay*, *hit rate*, *retrieval cost*, dan *cache-management cost* secara bersamaan untuk memahami *trade-off* antara performa dan efisiensi pengelolaan *cache*. Oleh karena itu, penelitian ini menggunakan metrik performa berupa *average latency* dan *p95 latency*, metrik *overhead* berupa *PIP calls* dan *cache hits*, metrik *freshness* berupa *stale cache hit rate*, serta metrik kualitas keputusan berupa *false allow rate* dan *false deny rate*. Untuk mengevaluasi dampak keterlambatan pencabutan hak akses, digunakan *revocation false allow rate* sebagai indikator risiko ketika akses tetap diizinkan setelah hak akses dicabut.

Dengan demikian, sistem ABAC berbasis *edge* tidak dapat dinilai aman hanya karena menghasilkan latensi rendah atau *cache hit* tinggi. Sistem yang cepat tetapi menghasilkan *false allow* dalam jumlah tinggi tetap tidak dapat dianggap aman karena keputusan otorisasinya gagal melindungi sumber daya. Oleh karena itu, evaluasi perlu menyeimbangkan performa, *overhead*, *freshness*, *revocation handling*, kualitas keputusan, serta beban *resource* seperti *CPU time* dan *peak memory*.

II.6 Optimasi dan Metaheuristik untuk Konfigurasi ABAC Edge

Konfigurasi ABAC berbasis *edge* dalam penelitian ini dipandang sebagai masalah optimasi multi-parameter karena melibatkan pemilihan atribut yang boleh di-*cache*, penentuan TTL per atribut, dan ukuran *cache*. Weerasinghe dkk. (2023) menunjukkan bahwa *adaptive context caching* pada aplikasi IoT perlu mengoptimalkan performa dan efisiensi biaya ketika sistem merespons *context query* secara *real-time*. Selain itu, penelitian tersebut juga menekankan bahwa *adaptive caching* mencakup proses *selection*, *refreshing*, *scaling*, dan *eviction* yang saling memengaruhi dalam manajemen siklus hidup *cache*. Medvedev dkk. (2023) menjelaskan bahwa *freshness*, *accuracy*, *completeness*, dan latensi merupakan parameter QoS, sedangkan biaya operasional perlu dipertimbangkan secara bersamaan dalam pengelolaan *context caching* berbasis IoT *middleware*. Dengan demikian, konfigurasi *cache* dan TTL tidak cukup ditentukan secara manual karena setiap parameter dapat memengaruhi latensi, *PIP calls*, *cache hit*, dan risiko penggunaan atribut yang usang.

Random search digunakan sebagai *baseline* optimasi karena pendekatan ini memilih konfigurasi secara acak dari ruang solusi yang telah ditentukan. Bergstra dan Bengio

(2012) menunjukkan bahwa *random search* dapat menjadi strategi yang lebih efisien dibandingkan *grid search* ketika hanya sebagian parameter yang memiliki pengaruh besar terhadap performa sistem. Selain itu, Feng dkk. (2025) menggunakan *random search* sebagai *baseline* pembandingan pada optimasi *multi-objective* berbasis GWO, bersama *Genetic Algorithm* (GA). Oleh karena itu, *random search* relevan digunakan sebagai *baseline* awal untuk mengevaluasi apakah algoritma metaheuristik mampu menemukan konfigurasi TTL, ukuran *cache*, dan *cacheable attribute* yang lebih optimal dibandingkan pemilihan konfigurasi secara acak.

GWO digunakan sebagai metode utama karena mampu melakukan pencarian berbasis *fitness* pada ruang solusi yang kompleks tanpa membutuhkan model analitik tertutup. Mirjalili dkk. (2014) menjelaskan bahwa GWO meniru hirarki sosial serigala abu-abu yang terdiri dari *alpha*, *beta*, *delta*, dan *omega*, di mana kandidat solusi terbaik memandu proses pembaruan posisi kandidat lain. Dalam GWO, proses optimasi dilakukan secara *derivation-free* dan berbasis pencarian stokastik sehingga cocok digunakan pada permasalahan optimasi dengan ruang pencarian yang kompleks dan sulit dimodelkan secara matematis. Dalam penelitian ini, mekanisme tersebut relevan karena setiap kandidat solusi dapat merepresentasikan kombinasi konfigurasi *cacheable attribute*, TTL per atribut, dan ukuran *cache*, sedangkan *fitness* dihitung berdasarkan gabungan metrik performa dan keamanan.

Relevansi GWO untuk konteks IoT dan *fog computing* diperkuat oleh penelitian terbaru pada *task scheduling* dan *resource optimization*. Alsadie dan Alsulami (2025) mengembangkan TS-GWO untuk *task scheduling* IoT pada lingkungan *fog-cloud* dan menunjukkan bahwa modifikasi GWO dapat meningkatkan kemampuan eksplorasi dan eksploitasi dalam menemukan solusi penjadwalan yang efisien. Alsamarai dan Uçan (2024) juga mengusulkan PC-GWO untuk optimasi performa dan biaya pada lingkungan *fog-cloud* dengan tujuan menurunkan *makespan*, konsumsi energi, dan biaya komputasi. Meskipun penelitian-penelitian tersebut menunjukkan efektivitas GWO pada optimasi *fog/edge*, penerapannya masih dominan pada *task scheduling* dan *resource allocation* dan bukan pada konfigurasi keamanan ABAC seperti TTL atribut, ukuran *cache*, dan pemilihan atribut yang boleh di-*cache*.

berdasarkan hal tersebut, metaheuristik digunakan karena konfigurasi *cache* dan TTL pada ABAC *edge* sulit ditentukan secara manual, terutama ketika tujuan optimasi mencakup performa dan keamanan secara bersamaan. Konfigurasi yang optimal tidak hanya harus menurunkan *average latency*, *p95 latency*, dan *PIP calls*, tetapi juga harus menekan *stale cache hit*, *false allow*, *false deny*, dan *revocation false allow*. Oleh karena itu, penggunaan GWO dalam penelitian ini diarahkan untuk

mengisi celah antara optimasi performa *fog/edge* yang telah banyak diteliti dan optimasi parameter keamanan ABAC yang masih relatif terbuka.

II.7 Dasar Pemilihan Konfigurasi Uji dan Skenario Workload

II.7.1 Dasar Pemilihan Konfigurasi Uji

Konfigurasi pengujian dalam penelitian ini disusun secara bertahap untuk memisahkan pengaruh penempatan otorisasi di *edge*, penggunaan *cache atribut*, strategi *freshness*, dan mekanisme optimasi konfigurasi. *Cloud-only* ABAC digunakan sebagai *baseline* dengan verifikasi atribut terpusat karena sebagian besar sistem IAM pada *edge* IoT masih bergantung pada verifikasi izin berbasis *cloud*. Wang dkk. (2025) menunjukkan bahwa banyak sistem *edge* masih melakukan autentikasi dan otorisasi melalui IAM berbasis *cloud* sehingga setiap komunikasi *edge* tetap perlu diverifikasi ke *cloud*. Pendekatan tersebut dapat meningkatkan ketergantungan terhadap koneksi *remote* dan membatasi kemampuan sistem dalam menyediakan manajemen akses berlatensi rendah pada perangkat *edge* IoT.

Edge-fresh ABAC digunakan untuk mengevaluasi dampak pemindahan proses otorisasi ke *edge* tanpa melibatkan *cache atribut* sehingga pengaruh penempatan *edge* dapat dipisahkan dari pengaruh *caching*. Kalaria dkk. (2024) menunjukkan bahwa kontrol akses berbasis *fog* membawa pengambilan keputusan dan informasi kebijakan lebih dekat ke *end nodes* untuk meningkatkan kemampuan adaptasi, ketersediaan sumber daya, dan mengurangi *overhead* waktu pemrosesan pada lingkungan IoT yang dinamis. Pendekatan ini juga dirancang untuk mengurangi masalah *overhead* komunikasi dan *single point of failure* yang umum muncul pada arsitektur kontrol akses terpusat berbasis *cloud*. Dengan demikian, konfigurasi *edge-fresh* diperlukan untuk membedakan manfaat yang berasal dari otorisasi *edge* itu sendiri dengan manfaat tambahan yang berasal dari *cache atribut*.

Static cache ABAC digunakan sebagai *baseline caching* karena *context caching* merupakan pendekatan yang umum digunakan untuk meningkatkan efisiensi respons sistem IoT yang harus menangani *context queries* secara *real-time*. Weerasinghe dkk. (2023) menunjukkan bahwa *adaptive context caching* dapat meningkatkan efisiensi performa dan efisiensi biaya pada platform manajemen konteks ketika merespons *context queries* dalam waktu mendekati *real-time*. Namun, Weerasinghe dkk. (2023) juga menekankan bahwa *context caching* merupakan suatu siklus proses yang melibatkan *selection*, *refreshing*, *eviction*, dan *refresh overhead* yang dapat meningkatkan biaya pengelolaan konteks secara signifikan apabila tidak dikontrol

dengan baik. Medvedev dkk. (2023) juga menjelaskan bahwa *freshness*, *accuracy*, *completeness*, dan latensi merupakan QoS yang perlu dipertimbangkan bersama biaya operasional dalam manajemen konteks IoT. Oleh karena itu, *static cache* ABAC digunakan untuk mengevaluasi *trade-off* dasar antara pengurangan latensi dan risiko *stale attribute* sebelum mekanisme *freshness-aware* diterapkan.

Adaptive TTL ABAC digunakan untuk mengevaluasi apakah mekanisme *freshness-aware caching* dapat menghasilkan *trade-off* yang lebih baik dibandingkan *static caching*. Weerasinghe dkk. (2023) menunjukkan bahwa *context refreshing* yang terlalu sering dapat menyebabkan adanya lonjakan biaya manajemen konteks akibat meningkatnya *retrieval* dan *overhead* pemrosesan, sedangkan *refresh* yang terlalu jarang dapat menurunkan validitas konteks yang tersimpan di *cache*. Medvedev dkk. (2023) juga menunjukkan bahwa konfigurasi kecepatan *refresh* memengaruhi keseimbangan antara biaya operasional dan QoS pada IoT *middleware*. Dengan demikian, *adaptive TTL* ABAC digunakan sebagai *baseline* heuristik untuk menguji apakah TTL yang disesuaikan berdasarkan karakteristik atribut dapat mengurangi *stale cache hit* tanpa menghasilkan *overhead retrieval* yang berlebihan.

Random search ABAC digunakan sebagai *baseline* optimasi untuk mengontrol klaim bahwa peningkatan performa benar-benar berasal dari mekanisme optimasi, bukan sekadar akibat eksplorasi ruang konfigurasi. Bergstra dan Bengio (2012) menunjukkan bahwa *random search* dapat menjadi *baseline* yang kuat dalam optimisasi *hyper-parameter* karena pada banyak kasus hanya sebagian kecil parameter yang memiliki pengaruh dominan terhadap performa sistem. Selain itu, Feng dkk. (2025) menggunakan *random search* sebagai *baseline* pembandingan terhadap GWO dan GA pada skenario optimisasi multi objektif. Oleh karena itu, *random search* ABAC digunakan untuk mengevaluasi apakah optimasi metaheuristik benar-benar mampu menemukan konfigurasi TTL, ukuran *cache*, dan *cacheable attribute* yang lebih efektif dibandingkan pemilihan konfigurasi secara acak.

GWO-optimized ABAC digunakan sebagai metode optimasi utama karena konfigurasi *cache* atribut pada *edge* ABAC membentuk masalah optimasi multi-parameter dengan *trade-off* yang saling bertentangan antara latensi, *freshness*, *overhead* komunikasi dan kualitas keputusan otorisasi. Mirjalili dkk. (2014) menjelaskan bahwa GWO menggunakan mekanisme pencarian berbasis hirarki untuk mengeksplorasi ruang pencarian secara iteratif tanpa membutuhkan model analitik tertutup. Feng dkk. (2025) menunjukkan bahwa GWO dapat diterapkan pada skenario optimisasi multi-objektif dan dibandingkan terhadap *random search* serta GA pada ruang pencarian yang kompleks. Selain itu, Alsadie dan Alsulami (2025) menunjukkan bahwa

modifikasi GWO pada lingkungan *fog-cloud* mampu meningkatkan kemampuan eksplorasi dan eksploitasi sehingga menghasilkan penjadwalan dengan *makespan* dan konsumsi energi yang lebih baik. Meskipun penelitian sebelumnya lebih banyak berfokus pada *task schedulling* dan alokasi sumber daya, pendekatan tersebut menunjukkan bahwa GWO sesuai digunakan pada permasalahan optimasi *edge/fog* yang memiliki ruang pencarian besar dan *trade-off* multi-parameter. Oleh karena itu, *GWO-optimized* ABAC digunakan untuk mengevaluasi apakah optimasi metaheuristik dapat meningkatkan keseimbangan antara performa *cache* dan kualitas keputusan otorisasi dibandingkan pendekatan *baseline* lainnya.

Dengan konfigurasi pengujian yang bertahap, hasil eksperimen tidak hanya digunakan untuk menunjukkan metode dengan performa terbaik, tetapi juga untuk mengisolasi kontribusi masing-masing komponen terhadap perubahan latensi, *PIP calls*, *stale cache hit*, *false allow*, *false deny*, dan *revocation false allow*. Pendekatan ini memungkinkan evaluasi dilakukan secara lebih beralasan dan terukur sehingga *trade-off* antara performa, *freshness*, dan reliabilitas keputusan otorisasi dapat dianalisis secara lebih jelas.

II.7.2 Dasar Penentuan Skenario Dinamika Workload

Skenario *workload* dalam penelitian ini dibagi menjadi empat tingkat dinamika, yaitu *LOW_DYNAMIC*, *MEDIUM_DYNAMIC*, *HIGH_DYNAMIC*, dan *VERY_DYNAMIC*. Pembagian ini digunakan sebagai *workload* sintetis terkontrol untuk mengevaluasi kemampuan ABAC berbasis *edge* dalam menghadapi peningkatan *request rate*, perubahan atribut, *request* kritis, kondisi *burst*, *revocation*, serta ketidaktersediaan PIP. Pendekatan bertingkat ini relevan karena *workload* IoT dan *edge-cloud* dapat menunjukkan *bursty arrivals*, *temporal variability*, *varying workload*, serta *non-stationary behavior* sehingga evaluasi tidak cukup dilakukan hanya pada kondisi stabil. Latifi dkk. (2025) menjelaskan bahwa lingkungan IoT bersifat dinamis dan dapat berubah dengan cepat, sedangkan Kovtun dkk. (2025) menunjukkan bahwa trafik IoT 5G dapat memiliki karakteristik trafik yang bersifat *batch-like*, terstruktur secara terkelompok, dan mengalami variasi temporal. Selain itu, Raju dkk. (2026) menunjukkan bahwa lingkungan *edge-cloud* menghadapi *deadline* tugas yang beragam, beban kerja yang berubah-ubah, kondisi sistem yang dinamis, serta beban kerja IoT yang mengalami lonjakan trafik secara tiba-tiba yang suka berubah-ubah selama *runtime*.

Pembagian *workload* tidak hanya didasarkan pada jumlah *request*, tetapi juga pada karakteristik dinamis atribut yang digunakan dalam evaluasi kebijakan ABAC.

Díaz dan Mendoza (2025) menjelaskan bahwa otorisasi pada IoT membutuhkan mekanisme yang fleksibel dan dapat beradaptasi terhadap interaksi dinamis dan kontekstual pada lingkungan IoT. Oleh karena itu, setiap skenario dalam penelitian ini mengombinasikan *inter-arrival time*, perubahan *request* kritis, perubahan *trust*, perubahan *anomaly*, perubahan kondisi jaringan, perubahan *firmware*, perubahan *role*, probabilitas *revocation*, serta probabilitas ketidaktersediaan PIP. Pendekatan ini juga sejalan dengan Raju dkk. (2026) yang memodelkan penjadwalan *edge-cloud* menggunakan keadaan *workload* yang berubah secara dinamis, intensitas *workload*, kondisi antrean, serta kondisi sumber daya yang heterogen untuk mengevaluasi pengambilan keputusan adaptif pada *runtime environment* yang bersifat dinamis.

Nilai numerik pada setiap parameter tidak dimaksudkan sebagai standar universal *workload* IoT, melainkan sebagai parameter simulasi yang dikontrol untuk membentuk gradasi tekanan sistem dari kondisi normal hingga ekstrem. *LOW_DYNAMIC* merepresentasikan kondisi stabil dengan *request* yang realtif renggang dan perubahan atribut yang rendah. *MEDIUM_DYNAMIC* merepresentasikan kondisi operasional umum dengan peningkatan *request rate* dan volatilitas atribut sedang. *HIGH_DYNAMIC* berfungsi sebagai *stress test* terhadap *cache*, TTL, dan PIP karena *request* datang lebih padat dan atribut berubah lebih cepat. *VERY_DYNAMIC* digunakan sebagai *robustness test* terhadap *bursty workload* dan kondisi *runtime* yang sangat dinamis dengan lonjakan *request* yang lebih tajam. Pendekatan ini sejalan dengan Raju dkk. (2026) yang memperlakukan karakteristik *workload*, *bandwidth*, kapasitas *node*, dan kondisi sumber daya sebagai parameter simulasi yang dikontrol alih-alih dianggap sebagai konstanta tetap dunia nyata untuk mengevaluasi penjadwalan adaptif pada lingkungan *edge-cloud*.

Inter-arrival time ditentukan secara bertahap untuk membentuk peningkatan intensitas *request* dari kondisi rendah hingga tinggi. Rentang yang lebih panjang pada *LOW_DYNAMIC* digunakan untuk merepresentasikan kondisi stabil dengan tekanan sistem yang rendah, sedangkan rentang yang lebih pendek pada *HIGH_DYNAMIC* digunakan untuk membentuk *request arrival* yang lebih padat. Pada *VERY_DYNAMIC*, *inter-arrival time* yang jauh lebih pendek saat *burst* digunakan untuk menguji ketahanan sistem terhadap adanya lonjakan beban kerja dan perubahan intensitas *request* yang bersifat *non-stationary*. Pendekatan ini relevan dengan Raju dkk. (2026) yang memodelkan pola *arrival* beban kerja menggunakan proses stokastik untuk merepresentasikan lonjakan beban kerja pada lingkungan *edge-cloud*.

Probabilitas *request* kritis ditingkatkan secara bertahap untuk menguji kemampuan kebijakan ABAC dalam menangani akses berisiko tinggi. Nilai probabilitas yang lebih rendah pada *LOW_DYNAMIC* merepresentasikan kondisi dominan normal, sedangkan probabilitas yang lebih tinggi pada *HIGH_DYNAMIC* dan *VERY_DYNAMIC* digunakan untuk mensimulasikan peningkatan proporsi akses sensitif atau berisiko tinggi. Pendekatan ini relevan karena lingkungan *edge* dengan beban kerja tinggi dapat mengalami peningkatan waktu tunggu dan pelanggaran SLA ketika intensitas beban kerja meningkat dan sistem harus menangani permintaan yang heterogen secara bersamaan (Raju dkk., 2026).

Variasi *trust* dan *anomaly* dinaikkan secara bertahap karena kedua atribut tersebut merepresentasikan kondisi perilaku yang dapat berubah cepat. Wang dkk. (2025) menunjukkan bahwa keputusan akses pada *edge-IAM* menjadi problematis apabila sistem mengasumsikan nilai atribut selalu mutakhir dalam lingkungan IoT yang dinamis dan terdistribusi. Oleh karena itu, *trust step* dan *anomaly step* yang lebih tinggi pada *HIGH_DYNAMIC* dan *VERY_DYNAMIC* digunakan untuk menguji apakah mekanisme *cache* dan TTL masih mampu menjaga kualitas keputusan ketika atribut perilaku berubah secara cepat selama *runtime*.

Network flip probability digunakan untuk merepresentasikan perubahan kondisi konektivitas yang dapat memengaruhi kemampuan sistem memperoleh atribut terbaru dari sumber atribut. Nilai *network flip* dibuat meningkat pada skenario yang lebih dinamis dan dinaikkan lebih tinggi saat *burst* karena trafik IoT dapat mengalami *bursty arrival*, *temporal variability*, dan fluktuasi intensitas komunikasi (Kovtun dkk., 2025). Selain itu, Raju dkk. (2026) juga menunjukkan bahwa kondisi sistem yang dinamis dan ketersediaan sumber daya yang heterogen merupakan karakteristik penting pada lingkungan *edge-cloud* yang dapat memengaruhi perilaku penjadwalan dan QoS. *PIP unavailability* juga dinaikkan secara bertahap untuk menguji ketergantungan sistem terhadap sumber atribut. Mao dkk. (2025) menjelaskan bahwa PDP memperoleh atribut subjek, objek, lingkungan, dan data kontekstual dari PIP untuk mendukung keputusan akses dinamis.

Firmware flip dan *role flip* dibuat lebih rendah dibandingkan *trust* dan *anomaly* karena keduanya relatif lebih stabil. Namun, kedua parameter tetap dimasukkan karena dampak keamanannya tinggi apabila dalam kondisi usang. Bakhshi dkk. (2024) menunjukkan bahwa kerentanan *firmware* merupakan ancaman penting pada IoT karena lemahnya perlindungan *firmware* dan keterbatasan pembaruan dapat membuka peluang kompromi perangkat. Wang dkk. (2025) juga menunjukkan bahwa keputusan otorisasi dapat menjadi tidak reliabel ketika atribut lokal tidak lagi

sinkron dengan kondisi aktual akibat keterlambatan sinkronisasi atau penggunaan *cache* yang sudah usang. Dengan demikian, *firmware* dan *role* digunakan untuk menguji kategori atribut yang relatif jarang berubah tetapi berisiko tinggi apabila perubahan tidak segera tercermin dalam keputusan akses.

Revocation probability dipilih untuk menguji risiko adanya *delay* revokasi akibat penggunaan atribut yang sudah usang. Nilai revokasi dibuat rendah karena pencabutan hak akses bukan merupakan *event* yang terjadi pada setiap *request*, tetapi dampaknya sangat kritis ketika terjadi. Wang dkk. (2025) menegaskan bahwa *local caching* pada *edge* dapat menimbulkan risiko keamanan ketika data yang telah diperbarui atau dicabut di *cloud* masih dianggap valid di *edge* akibat sinkronisasi yang tertunda. Oleh karena itu, peningkatan *revocation probability* pada *HIGH_DYNAMIC* dan *VERY_DYNAMIC* digunakan untuk menguji apakah mekanisme *cache* dan TTL tetap mampu mencegah akses tidak sah ketika *privilege* subjek berubah atau dicabut dalam kondisi *workload* yang semakin dinamis.

II.8 Penelitian Terkait

Penelitian terkait ABAC untuk IoT umumnya menekankan kemampuan model berbasis atribut dalam menyediakan kontrol akses yang lebih *fine-grained*, dinamis, dan *context-aware* dibandingkan model tradisional. Díaz dan Mendoza (2025) menjelaskan bahwa model otorisasi pada IoT menentukan aksi spesifik yang boleh dilakukan suatu entitas setelah proses autentikasi berlangsung. Sementara itu, Ullah dkk. (2023) membahas integrasi *blockchain* dengan ABAC sebagai pendekatan untuk meningkatkan keamanan dan kontrol akses pada lingkungan IoT. Namun, sebagian besar penelitian pada kelompok ini lebih banyak menekankan aspek ekspresivitas kebijakan, skalabilitas, desentralisasi, dan efisiensi mekanisme akses, sementara pembahasan mengenai *freshness* atribut sebagai faktor yang dapat memengaruhi reliabilitas keputusan otorisasi masih relatif terbatas.

Kelompok penelitian otorisasi *edge/fog* menunjukkan bahwa pemindahan proses pengambilan keputusan ke *edge* atau *fog* dapat mengurangi ketergantungan pada *cloud* dan mendukung respons yang lebih cepat. Kalaria dkk. (2024) mengusulkan kontrol akses adaptif berbasis *fog* yang *context-aware* dengan membawa proses pengambilan keputusan dan informasi kebijakan lebih dekat ke *end nodes*. Namun, penurunan latensi tidak secara otomatis menjamin keputusan akses tetap akurat apabila atribut yang digunakan pada *edge* sudah tidak mutakhir. Wang dkk. (2025) secara eksplisit menyatakan bahwa *edge-enabled IAM* yang mengasumsikan atribut selalu mutakhir dapat menghasilkan keputusan akses yang patut dipertanyakan pada

lingkungan IoT yang dinamis dan terdistribusi.

Penelitian tentang sinkronisasi atribut dan revokasi mulai menyoroti pentingnya konsistensi atribut dan pencabutan hak akses. Wang dkk. (2025) membahas LAA di *edge* dengan mempertimbangkan *freshness* atribut dan *trustworthiness* penyedia atribut. Selain itu Peñuelas-Angulo dkk. (2023) menegaskan bahwa revokasi merupakan permasalahan penting pada skema kontrol akses *fine-grained* berbasis atribut. Namun, studi-studi tersebut belum secara eksplisit membahas optimasi konfigurasi TTL atribut, ukuran *cache*, maupun pemilihan atribut yang layak di-*cache* sebagai bagian dari mekanisme *security-aware edge authorization*.

Penelitian *edge caching* umumnya menunjukkan bahwa *caching* dapat menurunkan latensi, meningkatkan responsivitas sistem, dan mengurangi biaya operasional pada lingkungan IoT. Weerasinghe dkk. (2023) membahas *adaptive context caching* untuk aplikasi IoT *real-time*. Sementara itu, Medvedev dkk. (2023) membahas *trade-off* antara *freshness*, latensi, QoS, dan biaya operasional dalam strategi *caching* dan *prefetching* pada IoT *middleware*. Namun, fokus utama penelitian tersebut masih berada pada *context/data caching*, bukan *attribute caching* untuk pengambilan keputusan keamanan berbasis ABAC.

Penelitian metaheuristik pada IoT dan *fog computing* juga menunjukkan efektivitas algoritma seperti GWO untuk penjadwalan tugas dan alokasi sumber daya. Alsadie dan Alsulami (2025) menggunakan GWO yang dimodifikasi untuk penjadwalan tugas pada *fog/cloud computing*, sedangkan Alsamarai dan Uçan (2024) menggunakan PC-GWO untuk meningkatkan performa penjadwalan, utilisasi sumber daya, dan efisiensi energi pada lingkungan *fog-cloud*. Akan tetapi, penggunaan GWO pada penelitian tersebut masih difokuskan pada optimasi performa komputasi, bukan optimasi parameter keamanan ABAC seperti TTL atribut, ukuran *cache*, dan *cacheable attribute*.

Berdasarkan tinjauan literatur tersebut, masih terbatas penelitian yang secara terpadu menggabungkan *edge-based ABAC*, *attribute caching*, *freshness-aware authorization*, optimasi berbasis metaheuristik, dan evaluasi berorientasi keamanan terhadap kualitas keputusan akses. Oleh karena itu, penelitian ini mengevaluasi konfigurasi ABAC berbasis *edge* menggunakan metrik performa, *overhead*, *freshness*, dan kualitas keputusan secara bersamaan. Dengan demikian, sistem tidak hanya dinilai dari sisi latensi atau *cache hit rate*, tetapi juga dari kemampuannya menjaga reliabilitas dan keamanan keputusan akses ketika atribut berubah secara dinamis.

Tabel II.1 Perbandingan penelitian terkait ABAC, caching, dan optimasi pada IoT

Penelitian	Kelompok	Domain	Edge/Fog	ABAC	Cache/Freshness	Optimasi	Metrik/Evaluasi	Keterbatasan
Díaz dan Mendoza (2025)	ABAC/ authorization IoT	IoT authorization survey	Tidak utama	Ya	Terbatas	Tidak	Analisis komparatif model dan arsitektur otorisasi	Membahas authorization IoT dan kebutuhan <i>fine-grained/context-aware</i> , tetapi freshness atribut belum menjadi fokus utama.
Ullah dkk. (2023)	ABAC untuk IoT	Blockchain-based ABAC for IoT	Tidak utama	Ya	Terbatas	Tidak	Security/privacy, latency, overhead, scalability	Fokus pada ABAC dan blockchain; belum membahas <i>stale attribute</i> dan optimasi TTL/cache.
Kalaria dkk. (2024)	Edge/fog authorization	Fog-based context-aware access control	Ya	Ya	Tidak utama	Tidak	Processing time overhead, security improvement	Fog menurunkan latency dan cloud dependency, tetapi belum fokus pada <i>stale attribute</i> /freshness atribut.
Wang dkk. (2025)	Edge-IAM; synchronization	Edge-enabled IAM IoT	Ya	Ya	Attribute freshness dan synchronization	Tidak	Freshness, source trustworthiness, decision reliability, overhead	Membahas freshness dan local authorization, tetapi belum mengoptimalkan TTL, cache size, dan cacheable attributes.
Peñuelas-Angulo dkk. (2023)	Revocation berbasis atribut	ABE revocation for fog-enabled IoT	Ya	Tidak langsung	Revocation, bukan cache freshness	Tidak	User revocation, attribute revocation, cost comparison	Membahas revocation, tetapi belum mengaitkannya dengan optimasi TTL/cache atribut untuk edge ABAC.
Weerasinghe dkk. (2023)	Context caching	IoT context caching/CMP	Tidak utama	Tidak	Context freshness	Adaptive/RL-based caching	Cost efficiency, QoS/QoC, freshness, latency, hit rate	Fokus pada context/data caching, bukan cache atribut keamanan ABAC.
Medvedev dkk. (2023)	IoT caching	IoT middleware/CMP caching	Tidak utama	Tidak	Freshness–latency–cost	Cost-based caching/prefetching	QoS, freshness, latency, operational cost	Membahas trade-off caching, tetapi bukan otorisasi ABAC.
Alsadie dan Alsulami (2025)	Metaheuristik IoT	Fog-cloud task scheduling	Ya	Tidak	Tidak	Modified GWO	Makespan, energy consumption	GWO efektif untuk scheduling, tetapi belum diarahkan ke optimasi parameter ABAC.
Alsamarai dan Uçan (2024)	Metaheuristik IoT	Fog-cloud task scheduling	Ya	Tidak	Tidak	PC-GWO	Makespan, energy, resource utilization, cost	Optimasi performa/resource allocation, bukan security-aware ABAC.
Penelitian ini	ABAC edge security-aware optimization	Edge IoT authorization	Ya	Ya	TTL per atribut, stale hit	Random Search, GWO	FAR, FDR, stale hit, revocation FAR, latency, PIP calls	Simulasi terkontrol; belum diuji pada deployment fisik.

II.9 Sintesis Literatur dan Posisi Penelitian

Berdasarkan tinjauan pustaka yang telah dibahas, ABAC merupakan model kontrol akses yang relevan untuk lingkungan IoT karena mampu mendukung keputusan akses yang bersifat *fine-grained* dan *context-aware*. ABAC memungkinkan keputusan akses tidak hanya ditentukan berdasarkan identitas atau peran, tetapi juga berdasarkan atribut subjek, objek, aksi, serta kondisi lingkungan. Namun, penerapan ABAC pada IoT masih menghadapi tantangan karena evaluasi kebijakan membutuhkan atribut yang akurat dan mutakhir. Semakin banyak atribut yang digunakan, semakin besar pula biaya pengambilan atribut, komunikasi dengan PIP, dan latensi keputusan akses.

Pemindahan evaluasi keputusan akses ke *edge* dapat mengurangi ketergantungan terhadap *cloud* dan mempercepat proses otorisasi. Akan tetapi, pendekatan ini juga menimbulkan persoalan baru, terutama ketika atribut disimpan secara lokal pada *cache gateway*. Atribut yang tersimpan di *cache* dapat menjadi tidak mutakhir ketika terjadi perubahan kondisi perangkat, perilaku akses, status jaringan, *firmware*, *role*, atau pencabutan hak akses. Kondisi tersebut dapat menyebabkan keputusan otorisasi yang tidak akurat, seperti *false allow*, *false deny*, dan *revocation false allow*.

Penelitian terdahulu telah membahas ABAC untuk IoT, otorisasi berbasis *edge/fog*, sinkronisasi atribut, *caching*, dan optimasi metaheuristik pada sistem IoT. Namun, masih terdapat celah penelitian pada optimasi *freshness* atribut dalam ABAC berbasis *edge*, khususnya dalam penentuan atribut yang boleh di-*cache*, nilai TTL per atribut, dan ukuran *cache* lokal secara terpadu dengan mempertimbangkan *trade-off* antara performa dan kualitas keputusan otorisasi.

Posisi penelitian ini berada pada irisan antara ABAC untuk IoT, otorisasi berbasis *edge*, manajemen *freshness* atribut, dan optimisasi berbasis metaheuristik. Berbeda dari pendekatan yang hanya berfokus pada pemindahan evaluasi ke *edge* atau penggunaan *cache* untuk menurunkan latensi, penelitian ini menempatkan *cache* atribut sebagai komponen keamanan yang dapat memengaruhi kualitas keputusan akses. Oleh karena itu, optimasi tidak hanya diarahkan untuk mempercepat evaluasi, tetapi juga untuk menekan risiko penggunaan atribut yang sudah usang.

Kebaruan penelitian ini terletak pada formulasi optimasi *freshness* atribut pada ABAC berbasis *edge* dengan pendekatan metaheuristik. Optimasi dilakukan terhadap *cacheable attribute*, TTL per atribut, dan ukuran *cache*, tanpa mengubah semantik kebijakan ABAC. Dengan demikian, seluruh atribut tetap dievaluasi dalam proses pengambilan keputusan, sedangkan metaheuristik hanya menentukan strategi akuisisi dan penyimpanan atribut yang paling sesuai.

Evaluasi penelitian dilakukan secara komparatif melalui beberapa konfigurasi uji, yaitu *cloud-only ABAC*, *edge-fresh ABAC*, *static cache*, *adaptive TTL*, *random search*, dan GWO. Selain itu, *workload* dibagi menjadi skenario rendah, sedang, tinggi, dan sangat dinamis untuk menguji ketahanan mekanisme terhadap perubahan *request rate*, perubahan atribut, *request* kritis, revokasi, dan ketersediaan PIP. Dengan rancangan tersebut, penelitian ini tidak hanya mengukur performa melalui latensi dan *PIP calls*, tetapi juga mengevaluasi kualitas keputusan akses melalui *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*.

Dengan demikian, Bab II ini menjadi dasar konseptual bagi Bab III, yang akan menjelaskan rancangan arsitektur sistem, model evaluasi ABAC berbasis *edge*, representasi solusi optimasi, fungsi *fitness*, konfigurasi uji, skenario *workload*, serta metrik evaluasi yang digunakan dalam penelitian.

BAB III METODOLOGI PENELITIAN

III.1 Jenis dan Pendekatan Penelitian

Penelitian ini menggunakan pendekatan *Design Science Research* (DSR) yang diperkuat dengan evaluasi eksperimental berbasis simulasi. Pendekatan DSR digunakan karena penelitian ini tidak hanya bertujuan menganalisis permasalahan kontrol akses pada lingkungan IoT, tetapi juga merancang artefak berupa mekanisme *GWO-optimized ABAC* untuk mengoptimalkan *cache* atribut dan *attribute freshness* pada ABAC berbasis *edge*. Artefak yang dirancang mencakup arsitektur ABAC berbasis *edge*, mekanisme *cache* atribut lokal pada *gateway*, formulasi optimasi *cacheable attribute*, nilai TTL per atribut, ukuran *cache*, serta fungsi *fitness* yang mempertimbangkan aspek performa dan kualitas keputusan akses.

Secara metodologis, penelitian ini bersifat kuantitatif eksperimental karena evaluasi dilakukan melalui pengukuran metrik numerik pada beberapa konfigurasi pengujian. Konfigurasi yang dibandingkan meliputi *cloud-only ABAC*, *edge-fresh ABAC*, *static cache ABAC*, *adaptive TTL ABAC*, *random search ABAC*, dan *GWO-optimized ABAC*. Perbandingan tersebut digunakan untuk memisahkan pengaruh penempatan otorisasi di *edge*, penggunaan *cache* atribut, strategi *freshness*, dan optimasi metaheuristik terhadap performa serta kualitas keputusan otorisasi.

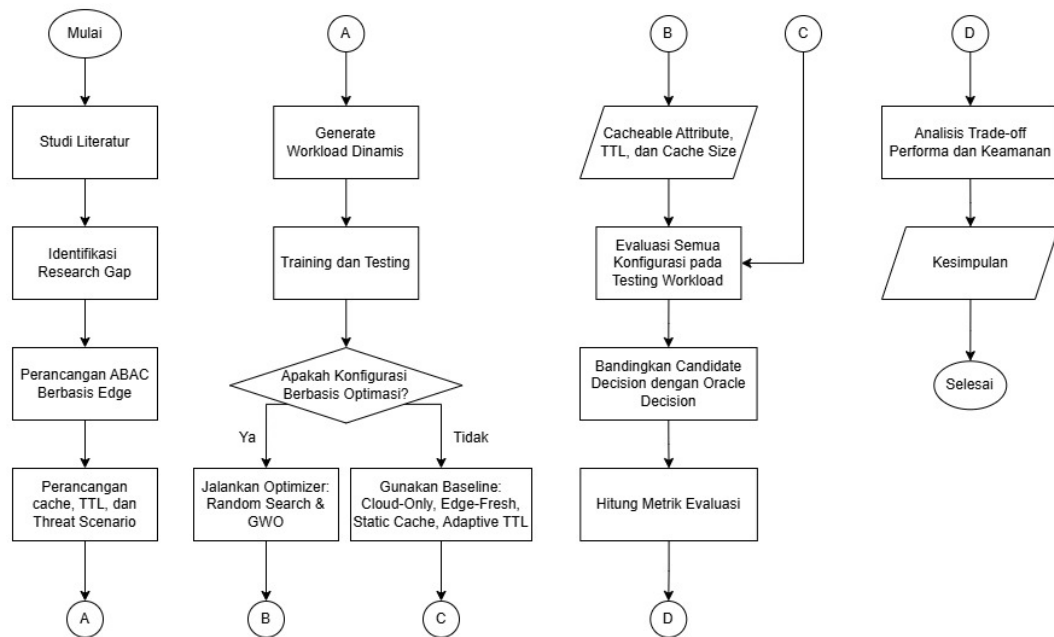
Eksperimen dilakukan melalui simulasi berbasis iFogSim2 dan implementasi Java untuk merepresentasikan lingkungan *edge* IoT dengan topologi *cloud-proxy-gateway-endpoint*. Simulasi dipilih karena memungkinkan pengaturan skenario secara terkontrol, termasuk jumlah *gateway* dan *endpoint*, dinamika *workload*, perubahan atribut, ketersediaan PIP, serta kejadian revokasi. Dengan demikian, setiap konfigurasi dapat diuji pada kondisi yang sama sehingga hasil perbandingan menjadi lebih objektif dan dapat direproduksi.

Evaluasi penelitian difokuskan pada dua aspek utama, yaitu efisiensi performa dan kualitas keputusan akses. Aspek performa diukur melalui *average latency*, *p95 latency*, jumlah *PIP calls*, *cache hits*, *CPU time*, dan penggunaan memori. Sementara

itu, kualitas keputusan akses diukur melalui *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation allow rate*. Dengan pendekatan tersebut, penelitian ini tidak hanya menilai konfigurasi yang paling cepat, tetapi juga menilai konfigurasi yang paling mampu menjaga keseimbangan antara efisiensi sistem, kesegaran atribut, dan keamanan keputusan otorisasi pada lingkungan IoT *edge* yang dinamis.

III.2 Alur Umum Penelitian

Secara umum, alur penelitian ini dapat digambarkan sebagai berikut:



Gambar III.1 Diagram alir alur umum penelitian

Penelitian ini dilaksanakan melalui tahapan sistematis yang dimulai dari studi literatur hingga penarikan kesimpulan berdasarkan hasil eksperimen. Alur penelitian dirancang tidak hanya untuk menguji kinerja mekanisme ABAC berbasis *edge*, tetapi juga untuk mengevaluasi kualitas keputusan otorisasi ketika sistem menggunakan mekanisme *cache* atribut dan pengaturan *freshness* melalui TTL. Oleh karena itu, tahapan penelitian ini mencakup proses perancangan model, pembangkitan *workload* dinamis, pemisahan data *training* dan *testing*, optimasi konfigurasi, evaluasi keputusan akses, serta analisis *trade-off* antara performa dan keamanan.

Tahap pertama adalah studi literatur. Pada tahap ini dilakukan kajian terhadap konsep keamanan IoT, model kontrol akses, ABAC, *edge/fog computing*, mekanisme *cache* atribut, *attribute freshness*, TTL, serta algoritma metaheuristik. Studi literatur

digunakan untuk memahami perkembangan penelitian terdahulu dan menemukan keterbatasan pendekatan yang telah ada, khususnya terkait evaluasi ABAC berbasis *edge* pada lingkungan IoT yang dinamis.

Tahap kedua adalah identifikasi *research gap*. Berdasarkan hasil studi literatur, ditemukan bahwa ABAC mampu mendukung kontrol akses yang *fine-grained* dan *context-aware*, tetapi implementasinya pada lingkungan IoT masih menghadapi tantangan berupa *overhead* pengambilan atribut, latensi evaluasi, serta kebutuhan atribut yang mutakhir. Pemindahan evaluasi ABAC ke *gateway edge* dapat mengurangi ketergantungan terhadap *cloud*, tetapi penggunaan *cache* atribut pada *edge* dapat menimbulkan risiko *stale attribute*. Kondisi tersebut dapat menyebabkan kesalahan keputusan akses, seperti *false allow*, *false deny*, dan *revocation false allow*.

Tahap ketiga adalah perancangan model ABAC berbasis *edge*. Pada tahap ini dirancang arsitektur sistem yang terdiri dari *endpoint IoT*, *gateway edge*, *proxy*, dan *cloud*. *Endpoint* menghasilkan permintaan akses, sedangkan *gateway edge* menjalankan proses evaluasi kebijakan ABAC. *Gateway* juga dilengkapi *cache* atribut lokal untuk mengurangi pengambilan atribut berulang dari PIP. Dalam rancangan ini, seluruh atribut tetap digunakan dalam evaluasi kebijakan ABAC. Dengan demikian, proses optimasi tidak mengubah semantik kebijakan akses, melainkan hanya mengoptimalkan strategi akuisisi atribut.

Tahap keempat adalah perancangan mekanisme *cache*, TTL dan *threat scenario*. Mekanisme *cache* digunakan untuk menyimpan atribut tertentu pada *gateway edge* dalam batas waktu tertentu. TTL menentukan masa berlaku atribut di *cache*. Apabila atribut masih valid, *gateway* dapat menggunakan atribut dari *cache*. Sebaliknya, apabila atribut tidak tersedia atau telah kedaluwarsa, *gateway* mengambil atribut aktual dari PIP. Pada tahap ini juga dirancang skenario ancaman yang relevan, terutama *stale attribute*, perubahan nilai atribut, pencabutan hak akses, dan ketidatersediaan PIP.

Tahap kelima adalah pembangkitan beban kerja dinamis. Beban kerja dibangkitkan untuk merepresentasikan permintaan akses pada lingkungan IoT dengan tingkat dinamika yang berbeda. Penelitian ini menggunakan empat skenario, yaitu *LOW_DYNAMIC*, *MEDIUM_DYNAMIC*, *HIGH_DYNAMIC*, dan *VERY_DYNAMIC*. Setiap skenario mencerminkan variasi laju permintaan, perubahan nilai *trust*, perubahan *anomaly score*, perubahan kondisi jaringan, perubahan status *firmware*, perubahan *role*, peluang revokasi, dan kemungkinan lonjakan permintaan akses.

Tahap keenam adalah pemisahan beban kerja menjadi data *training* dan *testing*.

Data *training* digunakan untuk proses pencarian konfigurasi oleh metode optimasi, sedangkan data *testing* digunakan untuk mengevaluasi seluruh konfigurasi secara adil. Pemisahan ini dilakukan untuk mencegah *data leakage*, yaitu kondisi ketika konfigurasi hasil optimasi diuji pada data yang sama dengan data yang digunakan saat proses optimasi.

Tahap ketujuh adalah penentuan jenis konfigurasi eksperimen. Pada tahap ini terdapat percabangan antara konfigurasi non-optimasi dan konfigurasi berbasis optimasi. Konfigurasi non-optimasi meliputi *cloud-only ABAC*, *edge-fresh ABAC*, *static cache ABAC*, dan *adaptive TTL ABAC*. Konfigurasi ini digunakan sebagai *baseline* untuk menilai pengaruh penempatan evaluasi, penggunaan *cache*, dan pengaturan TTL manual. Sementara itu, konfigurasi berbasis optimasi meliputi *random search* dan GWO. Pada jalur optimasi, metode optimasi mencari kombinasi atribut yang boleh *di-cache*, nilai TTL per atribut, dan ukuran *cache* lokal pada *gateway*.

Tahap kedelapan adalah evaluasi seluruh konfigurasi pada *testing workload*. Setelah konfigurasi *baseline* dan konfigurasi hasil optimasi diperoleh, seluruh metode diuji menggunakan data *testing* yang sama. Evaluasi dilakukan untuk memastikan bahwa perbandingan antar metode berlangsung secara adil karena setiap konfigurasi menghadapi pola *workload* dan perubahan atribut yang sama.

Tahap kesembilan adalah perbandingan *candidate decision* dengan *oracle decision*. *Oracle decision* merupakan keputusan ideal yang diperoleh berdasarkan atribut aktual atau *ground truth*. Sementara itu, *candidate decision* merupakan keputusan yang dihasilkan oleh konfigurasi yang sedang diuji, baik menggunakan atribut *fresh* maupun atribut dari *cache*. Perbandingan kedua keputusan tersebut digunakan untuk menentukan apakah terjadi *false allow* atau *false deny*.

Tahap kesepuluh adalah penghitungan metrik evaluasi. Metrik yang digunakan mencakup metrik performa dan metrik keamanan. Metrik performa meliputi *average latency*, *p95 latency*, jumlah *PIP calls*, *cache hits*, *CPU time*, dan *peak memory usage*. Metrik keamanan dan kualitas keputusan meliputi *false allow rate*, *false deny rate*, *stale cache hit rate*, dan *revocation false allow rate*. Penggunaan kedua kelompok metrik ini bertujuan agar evaluasi tidak hanya menilai kecepatan sistem, tetapi juga ketepatan keputusan otorisasi.

Tahap kesebelas adalah analisis *trade-off* antara performa dan keamanan. Pada tahap ini, hasil dari setiap konfigurasi dibandingkan untuk mengetahui sejauh mana *cache* dan optimasi TTL mampu menurunkan latensi dan *PIP calls* tanpa meningkatkan risiko keputusan akses yang salah. Konfigurasi dengan latensi paling rendah tidak

selalu dianggap baik apabila menghasilkan *false allow* atau *revocation false allow* yang tinggi. Oleh karena itu, analisis dilakukan dengan menempatkan aspek keamanan sebagai pertimbangan utama, terutama pada lingkungan IoT yang dinamis.

Tahap terakhir adalah penarikan kesimpulan. Kesimpulan disusun berdasarkan hasil evaluasi terhadap seluruh konfigurasi, hubungan hasil eksperimen dengan hipotesis penelitian, serta kemampuan pendekatan yang diusulkan dalam menyeimbangkan efisiensi, *freshness* atribut, dan kualitas keputusan otorisasi.

III.3 Arsitektur Sistem yang Diusulkan

Arsitektur sistem yang diusulkan dalam penelitian ini menggunakan pendekatan ABAC berbasis *edge* pada lingkungan IoT. Arsitektur ini dirancang untuk mendekatkan proses evaluasi keputusan akses ke *gateway edge*, sehingga *endpoint* IoT tidak perlu menjalankan proses otorisasi yang kompleks dan tidak seluruh permintaan akses harus selalu bergantung pada *cloud*. Secara umum, sistem terdiri atas empat komponen utama, yaitu *endpoint IoT*, *gateway edge*, *proxy/server*, dan *cloud/PIP*.

Endpoint IoT berperan sebagai penghasil permintaan akses. Setiap *request* membawa informasi yang dibutuhkan dalam evaluasi ABAC, seperti atribut subjek, objek, aksi, dan kondisi lingkungan. *Request* tersebut dikirimkan ke *gateway edge* untuk diproses. Pada penelitian ini, *endpoint* tidak menjalankan evaluasi kebijakan secara langsung, melainkan hanya mengirimkan permintaan akses kepada *gateway*. Hal ini dilakukan agar beban evaluasi kebijakan yang lebih kompleks dapat dipindahkan ke *node edge* yang memiliki sumber daya lebih besar dibanding *endpoint* IoT.

Gateway edge merupakan komponen utama dalam arsitektur yang diusulkan. Pada *gateway* ditempatkan PEP, PDP, dan *cache* atribut lokal. PEP menerima *request* dari *endpoint* dan meneruskannya ke PDP untuk dievaluasi. PDP bertugas mengevaluasi kebijakan ABAC berdasarkan atribut yang dibutuhkan. Setelah PDP menghasilkan keputusan akses, PEP menegakkan keputusan tersebut dalam bentuk *allow* atau *deny*, kemudian mengirimkan *decision response* kembali ke *endpoint*.

Cache atribut lokal pada *gateway* digunakan untuk menyimpan atribut tertentu yang dibutuhkan dalam evaluasi kebijakan. *Cache* ini bersifat lokal per *gateway*, sehingga atribut yang tersimpan pada satu *gateway* tidak dibagikan secara langsung ke *gateway* lain. Pendekatan ini digunakan untuk merepresentasikan kondisi *edge* yang terdistribusi, di mana setiap *gateway* memiliki *state cache* masing-masing. Penggunaan *cache* bertujuan untuk mengurangi jumlah pengambilan atribut dari PIP, menurunkan latensi evaluasi akses, dan mengurangi *overhead* komunikasi ke *cloud*.

Cloud/PIP berperan sebagai sumber atribut aktual atau *ground truth*. Apabila atribut tidak tersedia di *cache*, TTL atribut telah kedaluwarsa, atau atribut tersebut tidak dikonfigurasi sebagai atribut yang boleh di-*cache*, maka *gateway* akan mengirimkan *attribute query* ke *cloud/PIP* melalui *proxy/server*. *Cloud/PIP* kemudian mengembalikan nilai atribut aktual pada *gateway*. Selain menyediakan atribut aktual, *cloud/PIP* juga merepresentasikan sumber pembaruan atribut, termasuk perubahan status perangkat, perubahan *role*, perubahan kondisi *trust*, dan *revocation update*.

Proxy/server ditempatkan di antara *gateway edge* dan *cloud/PIP*. Komponen ini berfungsi sebagai perantara komunikasi antara *gateway* dan *cloud*. Dalam eksperimen, *proxy/server* merepresentasikan jalur komunikasi menengah yang dilalui ketika *gateway* harus mengambil atribut aktual dari *cloud/PIP*. Dengan adanya *proxy*, arsitektur dapat memodelkan adanya biaya komunikasi tambahan ketika evaluasi ABAC di *edge* membutuhkan atribut yang tidak tersedia secara lokal.

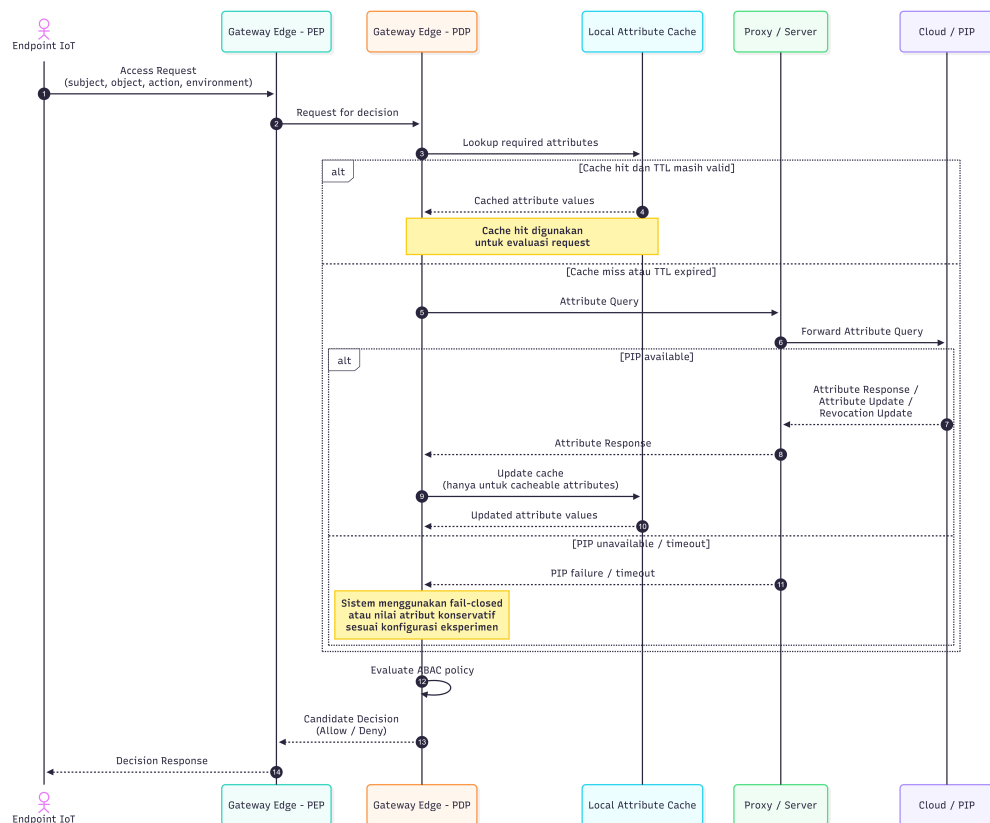
Prinsip penting dalam arsitektur ini adalah bahwa seluruh atribut yang dibutuhkan oleh kebijakan ABAC tetap dievaluasi. *Optimizer* tidak digunakan untuk menghapus atribut dari proses evaluasi kebijakan. Sebaliknya, *optimizer* hanya menentukan atribut mana yang boleh di-*cache*, nilai TTL per atribut, dan ukuran *cache* lokal pada *gateway*. Dengan demikian, optimasi yang dilakukan tidak melemahkan semantik kebijakan ABAC, melainkan hanya mengatur strategi akuisisi atribut agar evaluasi akses menjadi lebih efisien tanpa mengabaikan kualitas keputusan otorisasi.

Alur evaluasi akses dimulai ketika *endpoint IoT* mengirimkan permintaan akses ke PEP pada *gateway edge*. PEP kemudian meneruskan permintaan tersebut ke PDP. PDP memeriksa atribut yang dibutuhkan melalui *cache* atribut lokal. Jika atribut tersedia di *cache* dan TTL-nya masih valid, maka nilai atribut dari *cache* digunakan untuk evaluasi kebijakan. Kondisi ini disebut sebagai *cache hit*. Sebaliknya, jika atribut tidak tersedia di *cache* atau TTL telah kedaluwarsa, maka terjadi *cache miss* atau *TTL expired*. Pada kondisi tersebut, PDP mengirimkan *attribute query* ke *cloud/PIP* melalui *proxy/server* untuk memperoleh nilai atribut aktual.

Setelah *cloud/PIP* menerima *query*, *cloud/PIP* mengirimkan respons atribut kembali melalui *proxy/server* menuju *gateway*. Respons tersebut dapat berupa nilai atribut aktual, pembaruan atribut, atau informasi revokasi. Jika atribut tersebut termasuk dalam kategori *cacheable attribute*, *gateway* dapat memperbarui *cache* atribut lokal. Selanjutnya, PDP menggunakan atribut yang tersedia untuk mengevaluasi kebijakan ABAC dan menghasilkan *candidate decision*. Keputusan tersebut diteruskan ke PEP untuk ditegakkan, lalu PEP mengirimkan respons keputusan berupa *allow* atau *deny* kepada *endpoint*.

Arsitektur ini memungkinkan penelitian untuk mengevaluasi *trade-off* antara performa dan keamanan. Dari sisi performa, penggunaan *cache* lokal di *gateway* dapat menurunkan latensi dan jumlah *PIP calls*. Namun, dari sisi keamanan, penggunaan *cache* juga dapat menimbulkan risiko *stale attribute*, yaitu kondisi ketika nilai atribut di *cache* tidak lagi sesuai dengan kondisi aktual. Oleh karena itu, penelitian ini tidak hanya mengukur metrik performa, tetapi juga metrik kualitas keputusan.

Secara konseptual, arsitektur sistem yang diusulkan dan alur komunikasi antar komponennya dapat digambarkan melalui *sequence diagram* pada Gambar III.2.



Gambar III.2 Sequence diagram arsitektur sistem yang diusulkan

Gambar III.2 menunjukkan bahwa komunikasi antara *gateway edge* dan *cloud/PIP* bersifat dua arah. *Gateway* tidak hanya mengirimkan *attribute query*, tetapi juga menerima respons atribut, pembaruan atribut, atau pembaruan revokasi melalui *proxy/server*. Setelah atribut diterima, PDP dapat memperbarui *cache* untuk atribut yang dikonfigurasi sebagai *cacheable attributes*. Selanjutnya PDP mengevaluasi kebijakan ABAC dan mengirimkan *candidate decision* kepada PEP. PEP kemudian menegakkan keputusan tersebut dan mengembalikan respons keputusan kepada *endpoint IoT*. Dengan alur ini, arsitektur yang diusulkan dapat mengevaluasi manfaat

cache atribut lokal sekaligus mengukur risiko keamanan akibat atribut yang tidak mutakhir.

III.4 Model ABAC yang Digunakan

III.4.1 Komponen ABAC

III.4.2 Policy ABAC

III.4.3 Oracle Decision dan Candidate Decision

III.5 Model Cache Atribut dan Freshness

III.6 Threat Model dan Asumsi Keamanan

III.7 Desain Skenario Workload

III.8 Konfigurasi Eksperimen

III.8.1 Cloud-Only ABAC

III.8.2 Edge Fresh ABAC

III.8.3 Static Cache ABAC

III.8.4 Adaptive-TTL ABAC

III.8.5 Random Search ABAC

III.8.6 GWO-Optimized ABAC

III.9 Representasi Solusi Optimasi

III.10 Fungsi Fitness

III.11 Algoritma Optimasi

III.11.1 Random Search

III.11.2 Grey Wolf Optimization

III.12 Training dan Testing

III.13 Metrik Evaluasi

III.13.1 Metrik Performa

III.13.2 Metrik Kualitas Keputusan⁴³

III.13.3 Metrik Freshness dan Revocation

BAB IV HASIL DAN PEMBAHASAN

IV.1 Implementasi Simulasi pada iFogSim2

IV.2 Konfigurasi Lingkungan dan Skenario Pengujian

IV.3 Hasil Pengujian dan Analisis

IV.4 Pembahasan

BAB V KESIMPULAN DAN SARAN

V.1 Kesimpulan

V.2 Saran

DAFTAR PUSTAKA

- Afrin, S., Rafa, S. J., Kabir, M., Farah, T., Alam, M. S. B., Lameesa, A., Ahmed, S. F. dan Gandomi, A. H. (2025) : Industrial Internet of Things: Implementations, challenges, and potential solutions across various industries, *Computers in Industry*, **170**, 104317.
- Alsadie, D. dan Alsulami, M. (2025) : Modified grey wolf optimization for energy-efficient internet of things task scheduling in fog computing, *Scientific Reports*, **15**, 14730.
- Alsamarai, N. A. dan Uçan, O. N. (2024) : Improved Performance and Cost Algorithm for Scheduling IoT Tasks in Fog–Cloud Environment Using Gray Wolf Optimization Algorithm, *Applied Sciences*, **14**, 1670.
- Altaibek, M., Issainova, A., Aidynov, T., Kuttymbek, D., Abisheva, G. dan Nurusheva, A. (2025) : A Survey of Cross-Layer Security for Resource-Constrained IoT Devices, *Applied Sciences*, **15**, 9691.
- Ameer, S., Benson, J. dan Sandhu, R. (2023) : Hybrid Approaches (ABAC and RBAC) Toward Secure Access Control in Smart Home IoT, *IEEE Transactions on Dependable and Secure Computing*, **20**, 4032–4051.
- Azad, M. A., Abdullah, S., Arshad, J., Lallie, H. dan Ahmed, Y. H. (2024) : Verify and trust: A multidimensional survey of zero-trust security in the age of IoT, *Internet of Things*, **27**, 101227.
- Bakhshi, T., Ghita, B. dan Kuzminykh, I. (2024) : A Review of IoT Firmware Vulnerabilities and Auditing Techniques, *Sensors*, **24**, 708.
- Bakhtiary, V., Mirabi, M., Salajegheh, A. dan Erfani, S. H. (2024) : Combo-Chain: Towards a hierarchical attribute-based access control system for IoT with smart contract and sharding technique, *Internet of Things*, **25**, 101080.
- Bergstra, J. dan Bengio, Y. (2012) : Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* **13**, 281–305.
- Cremonezi, B., Da Rocha, L. F., Vieira, A. B., Nacif, J., De Oliveira, A. L. dan Silva, E. F. (2023) : A Bi-directional Attribute Synchronization Mechanism for Access Control in IoT Environments, *Mobile Computing, Applications, and Services*, 75–88.
- Cremonezi, B., Filho, A. R. G., Silva, E. F., Nacif, J. A. M., Vieira, A. B. dan Nogueira, M. (2022) : Improving the attribute retrieval on ABAC using opportunistic caches for Fog-Based IoT Networks, *Computer Networks*, **213**, 109000.

- Díaz, J. P. dan Mendoza, F. A. (2025) : Authorization models for IoT environments: A survey, *Internet of Things*, **29**, 101430.
- Feng, X., Wang, J., Sun, L., Zhang, C. dan Wang, T. (2025) : Dynamic Resource Allocation in Full-Duplex Integrated Sensing and Communication: A Multi-Objective Memetic Grey Wolf Optimizer Approach, *Electronics*, **14**, 3763.
- Ferraiolo, D. F., Sandhu, R., Gavrila, S., Kuhn, D. R. dan Chandramouli, R. (2001) : Proposed NIST standard for role-based access control, *ACM Transactions on Information and System Security*, **4**, 224–274.
- Frimpong, E., Michalas, A. dan Ullah, A. (2022) : Footsteps in the fog: Certificateless fog-based access control, *Computers & Security*, **121**, 102866.
- Hu, V. C., Ferraiolo, D., Kuhn, R., Schnitzer, A., Sandlin, K., Miller, R. dan Scarfone, K. (2014). *Guide to Attribute Based Access Control (ABAC) Definition and Considerations*. Tech. rep. NIST SP 800-162. National Institute of Standards dan Technology, NIST SP 800–162.
- Kalaria, R., Kayes, A. S. M., Rahayu, W., Pardede, E. dan Salehi Shahraki, A. (2024) : Adaptive context-aware access control for IoT environments leveraging fog computing, *International Journal of Information Security*, **23**, 3089–3107.
- Kokila, M. dan Reddy K, S. (2025) : Authentication, access control and scalability models in Internet of Things Security–A review, *Cyber Security and Applications*, **3**, 100057.
- Kovtun, V., Yukhimchuk, M., Alsayaydeh, J. A. J. dan Berdysheva, D. (2025) : Stochastic modelling of delays and buffering in 5G-IoT ecosystems with programmable P4 switches based on BMAP, *PLOS One*, **20**, e0330526.
- Krishnasrija, R., Mandal, A. K., Halder, R. dan Cortesi, A. (2026) : A Dynamic Context-Aware and Role-Capability Based Access Control Mechanism for Internet of Things, *Journal of Network and Systems Management*, **34**, 30.
- Laroui, M., Nour, B., Mounsla, H., Cherif, M. A., Afifi, H. dan Guizani, M. (2021) : Edge and fog computing for IoT: A survey on current research activities & future directions, *Computer Communications*, **180**, 210–231.
- Latifi, M., Derakhshanfard, N. dan Heydari, H. (2025) : Optimizing the distribution of tasks in Internet of Things using edge processing-based reinforcement learning, *Intelligent Systems with Applications*, **28**, 200585.
- Losty, A., Mishra, A. K., Cunche, M. dan Mandalari, A. M. (2025) : Towards Operational and Security Best Practices for DNS in the Internet of Things, *Proceedings of the Applied Networking Research Workshop on ZZZ*, 160–163.

- Mao, Y., Fu, W., Zhao, Y., Yuan, Z., Sun, Z. dan Zhao, Y. (2025) : A Zero-Trust Access Control Model Based on Attribute and Dynamic Trust Evaluation for Cloud Environments, *Symmetry*, **17**, 2059.
- Medvedev, A., Hassani, A., Belov, G., Weerasinghe, S., Huang, G.-L., Zaslavsky, A., Loke, S. W. dan Jayaraman, P. P. (2023) : Refresh Rate-Based Caching and Prefetching Strategies for Internet of Things Middleware, *Sensors*, **23**, 8779.
- Mirjalili, S., Mirjalili, S. M. dan Lewis, A. (2014) : Grey Wolf Optimizer, *Advances in Engineering Software*, **69**, 46–61.
- Park, J. dan Sandhu, R. (2004) : The UCON_{ABC} usage control model, *ACM Transactions on Information and System Security*, **7**, 128–174.
- Park, Y. dan Han, J. (2025) : Smart Home Advancements for Health Care and Beyond: Systematic Review of Two Decades of User-Centric Innovation, *J Med Internet Res*, **27**, e62793.
- Peñuelas-Angulo, A., Feregrino-Uribe, C. dan Morales-Sandoval, M. (2023) : Revocation in attribute-based encryption for fog-enabled internet of things: A systematic survey, *Internet of Things*, **23**, 100827.
- Ragothaman, K., Wang, Y., Rimal, B. dan Lawrence, M. (2023) : Access Control for IoT: A Survey of Existing Research, Dynamic Policies and Future Directions, *Sensors*, **23**, 1805.
- Raju, L. R., Reddy, M. V. K., Surukanti, S. R., Sudhakar, G., Subrahmanya Sarma M, V. V. dan Adepu, A. (2026) : IntelliScheduler: an edge-cloud computing environment hybrid deep learning framework for task scheduling based on learning, *Scientific Reports*, **16**, 11219.
- Shreyas, J., Reena S. Kharat, Rajesh N. Phursule, Bhujanga Rao Madamanchi, V., S. Rakshe, D., Gupta, G., Jawarneh, M., F., S. dan Raghuvanshi, A. (2025) : Chaotic grey wolf optimization based framework for efficient task scheduling in cloud fog computing, *Bulletin of Electrical Engineering and Informatics*, **14**, 2066–2076.
- State of IoT 2025: Number of Connected IoT Devices Growing 14% to 21.1 Billion Globally*, diperoleh melalui situs internet: <https://iot-analytics.com/number-connected-iot-devices/>. Diunduh pada tanggal 12 Mei 2026.
- Tall, A. M. dan Zou, C. C. (2023) : A Framework for Attribute-Based Access Control in Processing Big Data with Multiple Sensitivities, *Applied Sciences*, **13**, 1183.
- Ullah, S. S., Oleshchuk, V. dan Pussewalage, H. S. G. (2023) : A survey on blockchain envisioned attribute based access control for internet of things: Overview, comparative analysis, and open research challenges, *Computer Networks*, **235**, 109994.

- Waheed, U., Khan, S. A., Masud, M., Jamshed, H., Jumani, T. A. dan Malik, N. U. R. (2025) : Blockchain-Based, Dynamic Attribute-Based Access Control for Smart Home Energy Systems, *Energies*, **18**, 1973.
- Wang, Y., Martínez-Ortega, J.-F., Castillojo, P. dan San Emeterio De La Parte, M. (2025) : Edge-enabled IAM for IoTs with edge-based access management and context-driven syncservice, *Journal of Systems Architecture*, **165**, 103430.
- Weerasinghe, S., Zaslavsky, A., Loke, S. W., Hassani, A., Medvedev, A. dan Abken, A. (2023) : Adaptive Context Caching for IoT-Based Applications: A Reinforcement Learning Approach, *Sensors*, **23**, 4767.
- Williams, P., Dutta, I. K., Daoud, H. dan Bayoumi, M. (2022) : A survey on security in internet of things with a focus on the impact of emerging technologies, *Internet of Things*, **19**, 100564.
- Xue, T., Zhang, Y., Wang, Y., Wang, W., Li, S. dan Zhang, H. (2025) : Edge computing for IoT: Novel insights from a comparative analysis of access control models, *Computer Networks*, **270**, 111468.
- Yu, H., Yuchi, X., Yang, X., Li, H., Yang, X. dan Wang, W. (2025) : DNS-Sensor: A Sensor-Driven Architecture for Real-Time DNS Cache Poisoning Detection and Mitigation, *Sensors*, **25**, 6884.